PhishStudy: A Simulation Platform for Measuring and Reducing Phishing Susceptibility

submitted in fulfilment of the requirements
for Major Project-1, Semester VII

BE Honors

By

Shetty Vikyath Vishwanath (Roll No: 5)
Mehtar Aman Yasin (Roll No: 6)
Singh Yash Anil (Roll No: 7)
Singh Abhishek Shailesh (Roll No: 8)

Under the guidance of
Dr. Yogita Shelar

Department of Information Technology

Vidyavardhini's College of Engineering & Technology, Vasai (W)
(Affiliated with University of Mumbai)

(2026-2027)

Declaration

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Shetty Vikyath Vishwanath (Roll No: 5)

Mehtar Aman Yasin (Roll No: 6)

Singh Yash Anil (Roll No: 7)

Singh Abhishek Shailesh (Roll No: 8)

Date: 21 October 2026

Abstract

Phishing remains one of the most effective ways to breach an organisation. Technical controls such as sender authentication, filtering and multi-factor authentication reduce volume and consequence, but the final decision is still made by a person under time pressure. This project designs, implements and evaluates PhishStudy, a self-hosted simulation platform that measures and helps reduce phishing susceptibility. The platform models the complete campaign lifecycle, issues a unique token to every recipient, records open, click, submit and report events without storing submitted values, computes funnel, department, lure and risk metrics, and closes the loop with immediate coaching and an eight-item awareness quiz. It is implemented using the Python standard library and SQLite, with a hardened administrative console that uses PBKDF2-SHA256 password hashing, hashed session tokens, per-session CSRF protection, login throttling, output escaping and an audit log. The evaluation uses a deterministic synthetic organisation of 24 employees in six departments, six lures and five launched campaigns (104 simulated deliveries). The platform records a 43.3% click rate, a 22.1% submission rate and a 19.2% reporting rate. Sales and Finance are the most exposed departments, while IT records no clicks and a 62.5% reporting rate. On the repeated M365 lure, the post-training campaign records an 8.3% submission rate and a 50.0% reporting rate against a baseline of 33.3% and 8.3%, and the quiz mean rises from 3.25/8 to 6.21/8. The synthetic study validates the measurement pipeline and the analytics; it does not establish causal effects in a real workforce. The report documents the design, implementation, 24 automated tests, nine security probes, performance measurements, limitations and a plan for a consented field pilot.

Index

Chapter No.	Particulars	Pg. No.
Chapter 1	Introduction	1
	1.1 Background	1
	1.2 Motivation	1
	1.3 Problem Statement	2
	1.4 Aim and Objectives	2
	1.5 Scope and Boundaries	3
	1.6 Contributions	4
	1.7 Organisation of the Report	4
Chapter 2	Review of Literature	6
	2.1 Review Method	6
	2.2 Phishing: Definitions and Attack Lifecycle	6
	2.3 Human Factors in Susceptibility	7
	2.4 Awareness Interventions	8
	2.5 Simulation Platforms and Measurement	9
	2.6 Technical Controls and the Human Layer	10
	2.7 Ethics and Legal Considerations	11
	2.8 Research Gap	11
	2.9 Summary	12
Chapter 3	Proposed System	13
	3.1 System Overview	13
	3.2 Design Goals	13
	3.3 System Architecture	14
	3.4 Functional Requirements	15
	3.5 Non-Functional Requirements	16
	3.6 Data Model	16
	3.7 Process Design	17
	3.8 Use Cases	19
	3.9 Threat Model and Safeguards	20
	3.10 Design Trade-offs and Summary	20
Chapter 4	Methodology	22
	4.1 Research Design	22
	4.2 Simulation Model	23
	4.3 Campaign Lifecycle and Experimental Procedure	24
	4.4 Metrics and Operational Definitions	24
	4.5 Composite Risk Score	25
	4.6 Training Design	26
	4.7 Data Collection, Consent and Privacy	26
	4.8 Validation Approach	27
	4.9 Ethical Framework	27

	4.10 Summary	28
Chapter 5	Experimental Setup	29
	5.1 Environment	29
	5.2 Synthetic Organisation	29
	5.3 Campaign Schedule	30
	5.4 Lure Catalogue	30
	5.5 Quiz Instrument	31
	5.6 Experimental Procedure	32
	5.7 Variables and Operationalisation	32
	5.8 Hardware and Software	32
	5.9 Summary	33
Chapter 6	Implementation Plan	34
	6.1 Development Approach	34
	6.2 Work Breakdown Structure	34
	6.3 Schedule	35
	6.4 Risk Management	36
	6.5 Quality Assurance	36
	6.6 Plan for Semester VIII	36
	6.7 Summary	37
Chapter 7	Implementation	38
	7.1 Technology Stack	38
	7.2 Package Structure	38
	7.3 Security Implementation	39
	7.4 Campaign Engine	40
	7.5 Tracking and Analytics	41
	7.6 Training Module	41
	7.7 Reporting and Exports	42
	7.8 Principal Screens	43
	7.9 Implementation Challenges	49
	7.10 Summary	50
Chapter 8	Testing and Validation	51
	8.1 Test Strategy	51
	8.2 Automated Test Suites	51
	8.3 Route and Flow Validation	52
	8.4 Security Probes	52
	8.5 Data Integrity and Reproducibility	53
	8.6 Performance	54
	8.7 Usability Review	54
	8.8 Defects Found and Fixed	55
	8.9 Limitations of the Validation	55
	8.10 Summary	56
Chapter 9	Results and Discussions	57
	9.1 Overview	57

	9.2 Campaign Outcomes	58
	9.3 Department Analysis	59
	9.4 Lure Difficulty	60
	9.5 Repeat Behaviour and Time-to-Event	61
	9.6 Composite Risk	62
	9.7 Training Outcomes	63
	9.8 Discussion	65
	9.9 Threats to Validity	65
	9.10 Summary	66
Chapter 10	Conclusions	67
	10.1 Summary of Work	67
	10.2 Conclusions	67
	10.3 Contributions	68
	10.4 Limitations	69
	10.5 Future Work	69
	10.6 Closing Remarks	70
A	Appendix A: Route Map	71
B	Appendix B: Quiz Instrument	72
C	Appendix C: Export Schema	73
	Plagiarism Report	77
	Literature Cited	77
	Acknowledgements	77

List of figures

Figure / Table No.	Description	Page
3.1	Layered architecture of PhishStudy. Every module reaches the database through the persistence layer, and every request passes through the security module.	15
3.2	Entity–relationship diagram. Foreign keys enforce referential integrity; deleting an employee cascades to targets and nulls quiz references.	16
3.3	Level-0 data-flow diagram. Events flow from the participant to the campaign engine and thence to analytics and training.	18
3.4	Campaign lifecycle. Targets are materialised at launch so that the denominator is stable for the whole campaign.	18
3.5	Use-case diagram for the administrator, the participant and the scheduled system actor.	19
4.1	Anatomy of a phishing attack, adapted from the campaign phases modelled by the platform. The platform instruments planning, delivery, interaction, capture and feedback; it stops short of exploitation.	24
6.1	Work completed and planned. The Semester VII prototype and synthetic pilot are complete; the Semester VIII field pilot and longitudinal evaluation are planned.	35
7.1	Administrator sign-in. The demo credentials are shown for the laboratory environment; a production deployment would remove them.	39
7.2	Dashboard: KPI cards, funnel, mutually exclusive outcome breakdown, campaign trend and the highest-risk employees.	40
7.3	Campaign list with lifecycle state, recipient count and per-campaign rates.	44
7.4	Campaign detail for the post-training M365 lure, showing the funnel and the per-target event table.	45
7.5	Lure catalogue. Each template carries a category, difficulty, sender identity and a list of red flags used by the coaching page.	45
7.6	People directory with department filters, consent state and the composite risk band.	46
7.7	Analytics workspace: department comparison, difficulty breakdown, template performance and the longitudinal trend.	46
7.8	Training dashboard: attempts, mean score, pass rate, pre/post comparison and item-level difficulty.	47
7.9	Participant inbox. Legitimate messages and simulated phishing messages share the same visual language.	47
7.10	Rendered M365 lure. The misspelled sender domain is visible in the message header; the body uses fear and urgency.	48
7.11	Look-alike landing page. The form is fictional and the submitted values are discarded.	48
7.12	Coaching page. The participant sees the red flags in the exact message they received, followed by a route to the quiz.	49
9.1	Delivery funnel. Reporting is an independent positive behaviour, not a later stage of the delivery funnel.	57
9.2	Campaign-level click, submission and reporting rates. The post-training M365 campaign reverses the baseline pattern.	58

9.3	Outcomes by department across all launched campaigns. Sales and Finance combine high click rates with high submission rates; IT combines zero clicks with high reporting.	59
9.4	Baseline outcomes by lure difficulty. The ordering is counter-intuitive and is explained by confounding between difficulty, lure and campaign timing.	61
9.5	Distribution of composite risk bands. The score is a triage tool, not a psychometric measurement.	62
9.6	Mean score and item-level accuracy before and after coaching. The post-training round improves on every item.	62

List of Tables

Figure / Table No.	Description	Page
2.1	Phishing channels and their characteristics	7
2.2	Comparison of representative platforms and studies	10
3.1	Functional requirements	15
3.2	Non-functional requirements and design response	16
3.3	STRIDE threat model and safeguards	18
4.1	Operational definitions of the reported metrics	24
4.2	Risk bands and recommended action	26
5.1	Composition of the synthetic population	29
5.2	Campaign schedule	30
5.3	Lure catalogue	31
5.4	Quiz blueprint	31
5.5	Hardware and software environment	33
6.1	Work breakdown structure	35
6.2	Milestones and acceptance criteria	35
6.3	Risk register	36
7.1	Package structure and responsibilities	39
7.2	Security controls implemented in the platform	40
8.1	Automated test suites	52
8.2	Security probes and results	53
8.3	Response times at two dataset sizes (milliseconds)	54
8.4	Defects found during testing and their fixes	55
9.1	Delivery funnel	57
9.2	Campaign-level outcomes	58
9.3	Outcomes by department	59
9.4	Baseline outcomes by lure difficulty	61
9.5	Employees with the highest repeat-click counts	62
9.6	Distribution of composite risk bands	62
9.7	Quiz results before and after training	63
9.8	Item-level accuracy before and after coaching	64
A.1	HTTP route map	71
B.1	Eight-item awareness quiz	72
C.1	Export schema	73
P.1	Plagiarism verification record	74

Chapter 1: Introduction

1.1 Background

Organisations of every size now depend on email, collaboration platforms and cloud identity providers to conduct routine business. That dependence has made the human inbox a permanent attack surface. Phishing is the use of a deceptive message, usually but not exclusively email, to persuade a person to disclose credentials, open a malicious attachment, transfer money or approve an attacker-initiated action [1]. The attack succeeds by exploiting trust, habit and authority rather than a software defect, which is why it survives repeated improvements in spam filtering and endpoint protection.

The scale of the problem is well documented. Verizon's 2025 Data Breach Investigations Report attributes a human element to approximately 60% of breaches [2], while the Anti-Phishing Working Group continues to record phishing volumes far above the levels seen before 2020 [3]. The IBM Cost of a Data Breach Report puts the global average cost of a breach at USD 4.44 million in 2025 [4], and the United States Federal Trade Commission reports that consumers lost USD 12.5 billion to fraud in 2024, a 25% increase over the previous year [5]. Phishing and spoofing remain among the most frequently reported categories in the FBI's Internet Crime Report [6]. These figures are not merely financial: they represent lost time, eroded trust and, in healthcare and critical infrastructure, risk to safety.

Technical controls reduce exposure but cannot remove the problem. Sender authentication through SPF, DKIM and DMARC, link rewriting, attachment sandboxing, multi-factor authentication and endpoint detection all raise the cost of an attack [7]. Attackers respond by registering look-alike domains, abusing legitimate file-sharing services, hijacking existing email threads and deploying adversary-in-the-middle proxies that relay one-time codes in real time [8]. The result is a moving target in which the final decision is still made by a person under time pressure. Security awareness is therefore not an alternative to engineering controls; it is the layer that determines whether a convincing message is reported, ignored or actioned.

1.2 Motivation

Awareness programmes are frequently delivered as an annual slide deck, a poster campaign or a compliance module that employees click through without engagement. Such programmes are

difficult to evaluate. An organisation may be unable to answer basic questions: what fraction of staff would submit credentials to a convincing login page, which departments are most exposed, whether reporting behaviour improves after coaching, and whether the same individuals are repeatedly caught. Without measurement, awareness spending becomes an act of faith.

Commercial phishing simulation platforms answer part of this need, but they carry licensing costs, require cloud connectivity and place employee behaviour data with a third party. Open-source tools such as GoPhish and King Phisher remove the licence fee but focus on campaign delivery rather than an integrated training and risk-analytics workflow [9], [10]. Small enterprises, colleges and public institutions often lack the staff to assemble these components into a defensible programme. There is a practical gap for a self-hosted, auditable platform that a small team can operate and explain.

There is also a methodological gap. Evaluating a phishing-awareness intervention with real employees raises consent, privacy and industrial-relations questions. A platform that first demonstrates its measurement logic on a transparent synthetic population, and only then moves to a consented field pilot, offers a lower-risk route to evidence. The synthetic population is not a substitute for field data; it is a testbed for the instrument, the analytics and the ethics safeguards.

1.3 Problem Statement

The problem addressed by this project is the absence of an affordable, privacy-preserving and technically credible way to measure and reduce phishing susceptibility in a small or medium-sized organisation. Existing approaches tend to be either expensive commercial services with limited transparency, or campaign scripts that record clicks without closing the loop through coaching, assessment and individual risk communication. A usable solution must model the complete campaign lifecycle, record behavioural events without capturing secrets, convert those events into actionable aggregate and individual insight, and provide immediate education at the moment of failure.

1.4 Aim and Objectives

The aim of this project is to design, implement and evaluate a simulation-based framework for measuring and reducing phishing susceptibility in organisations. The objective is not to build

another bulk mailer, but to build a defensible instrument that connects campaign design, behavioural telemetry, analytics, training and reporting in one auditable system. The specific objectives are:

1. To review the literature on phishing, human susceptibility, awareness training and simulation platforms, and to identify the requirements of a measurement-oriented system.
2. To specify the functional and non-functional requirements, architecture, data model and security controls of such a system.
3. To implement a working platform with a hardened administrative console, a campaign engine, per-recipient tracking, analytics, a coaching and quiz module, and exportable reporting.
4. To validate the platform through automated unit, route, security and data-integrity tests, together with a performance and usability review.
5. To evaluate behavioural outcomes in a controlled synthetic organisation and to derive practical implications for awareness programmes.
6. To document the limitations of the evaluation and to define a realistic plan for a consented field deployment in the next semester.

1.5 Scope and Boundaries

The scope of the present work is a single-tenant web application that simulates an organisational email environment. It includes campaign authoring and lifecycle control, a catalogue of lures, a participant inbox and mail renderer, tokenised landing pages, event tracking, analytics, risk scoring, a training quiz, a printable report and CSV/JSON exports. It is implemented using the Python standard library and SQLite and runs as a single process on commodity hardware.

The following are explicitly out of scope for this semester: delivery of real email over SMTP, collection or storage of real credentials, integration with a production identity provider, mobile applications, generative artificial intelligence for lure creation, and formal certification against ISO 27001 or SOC 2. The evaluation uses a deterministic synthetic organisation of 24 employees and 104 simulated deliveries across five launched campaigns. Consequently, the reported percentages describe the behaviour of that synthetic cohort and the behaviour of the

analytics pipeline; they are not population estimates for a real workforce. This boundary is deliberate and is revisited in the threats-to-validity discussion in Chapter 9.

1.6 Contributions

The project makes six contributions to the practice of phishing-awareness engineering:

- **An integrated, dependency-free platform.** Campaign delivery, participant experience, analytics, training and reporting are combined in one Python package that can be audited end to end and deployed without a container or a cloud account.
- **Privacy-preserving telemetry.** Landing-page submissions record only the names of the fields that were filled, never their values. Session tokens are stored as hashes, and the public participant flow does not require an account.
- **A transparent composite risk model.** Click, submission, reporting and quiz performance are combined into a bounded score with explicit bands, so that the ranking can be explained to an employee rather than presented as an opaque vendor metric.
- **A closed training loop.** A failed simulation leads to an immediate coaching page and an eight-item quiz; the same M365 lure is repeated after training so that the change in behaviour can be observed on a like-for-like stimulus.
- **A reproducible evaluation artefact.** A fixed-seed synthetic organisation, a documented outcome model and an export of every reported number make the results reproducible by another student or reviewer.
- **An ethics-by-design demonstration.** The system proves that realistic behavioural measurement can be performed without sending real mail, collecting real secrets or exposing individuals to punitive consequences.

1.7 Organisation of the Report

Chapter 2 reviews the literature on phishing, human factors, awareness training and simulation platforms, and states the research gap. Chapter 3 presents the proposed system: design goals, architecture, requirements, data model, process design, use cases and threat model. Chapter 4 explains the research design, the simulation model, the metric definitions and the ethical framework. Chapter 5 describes the experimental setup, including the synthetic population, campaign schedule, lure catalogue and quiz instrument. Chapter 6 presents the implementation

plan and the semester-wise schedule. Chapter 7 documents the implementation, including the security architecture and the principal screens. Chapter 8 reports testing and validation. Chapter 9 presents the results and discusses their meaning, limitations and threats to validity. Chapter 10 states the conclusions and the plan for future work. The appendices contain the route map, the quiz instrument and the export schema; the literature cited follows the IEEE numeric style.

Chapter 2: Review of Literature

This chapter reviews the body of work that informs the design of the platform. It begins with definitions and the anatomy of a phishing attack, then examines why individuals respond to deceptive messages, what awareness interventions have been shown to work, and how simulation platforms measure behaviour. It closes with the ethical and legal constraints that shape any realistic study, and with the gap that this project addresses.

2.1 Review Method

Sources were identified through IEEE Xplore, the ACM Digital Library, ScienceDirect, Google Scholar and the public reports of security vendors, regulators and standards bodies. Search terms included combinations of phishing, social engineering, susceptibility, awareness training, phishing simulation, embedded training, security behaviour and risk score. The review prioritised peer-reviewed empirical studies and technical surveys published between 2006 and 2026, supplemented by current industry reports where up-to-date prevalence or cost data were required. Detection-only machine learning papers were included only where they illuminated the limits of technical filtering. The sources cited in this report span empirical studies, surveys, standards and current industry reports.

2.2 Phishing: Definitions and Attack Lifecycle

Phishing is a form of social engineering in which an attacker impersonates a trusted entity in order to induce a target to perform an action that benefits the attacker. Aleroud and Zhou define it as a fraudulent process that combines technical deception with social persuasion to acquire sensitive information [1]. Chiew et al. broaden the definition to include the delivery of malware and the manipulation of financial transactions [11]. The common thread is that the message, not a software vulnerability, is the exploit.

A phishing campaign can be described as a sequence of phases. **Planning and targeting** selects a population and gathers naming conventions, brands, suppliers and organisational relationships. **Lure construction** creates a pretext, a sender identity, a message body and a landing page that mimics a legitimate service. **Delivery** distributes the message through email, SMS, voice, QR codes or a collaboration platform. **Interaction** moves the target from message to link, attachment or reply. **Capture** records credentials, session cookies, payment details or

malware execution. **Exploitation** uses the captured material to access accounts, move laterally, commit fraud or deploy ransomware. **Monetisation and clean-up** convert access into revenue and remove evidence. The platform in this project models the first five phases in a safe, instrumented form.

Phishing is not a single technique. Bulk phishing sends a generic lure to many recipients; spear phishing tailors the message to a small group or individual; whaling targets senior executives; clone phishing copies a genuine message and replaces its links; thread hijacking continues an existing conversation from a compromised mailbox; business email compromise (BEC) impersonates an executive or supplier to authorise a payment; and callback phishing uses a benign-looking message to persuade the target to telephone a fraudulent help desk. Multi-channel attacks combine email with SMS or voice to increase credibility. Table 2.1 summarises the principal channels.

Channel	Typical delivery	Representative lure	Why it works
Email	Bulk or targeted message with link or attachment	Password expiry, invoice, shared document	Ubiquitous, asynchronous, easy to spoof
Smishing	SMS or messaging app	Parcel delivery, bank alert, toll charge	Personal device, short message, strong urgency
Vishing	Telephone or voice note	Help-desk callback, fraud investigation	Real-time pressure, authority of a live voice
Quishing	QR code in email or physical poster	Parking fine, menu, MFA enrolment	Opaque destination, mobile browser
BEC	Compromised or look-alike mailbox	Payment instruction, gift-card request	Internal authority, existing trust
Adversary-in-the-middle	Reverse-proxy landing page	M365 or SSO login	Relays password and OTP in real time

Table 2.1: *Phishing channels and their characteristics*

The technical sophistication of the landing page matters, but the decisive variable is often the pretext. A message that arrives at the right moment, from a plausible sender, with a consequence that feels immediate, can succeed even when the target has been trained to look for spelling errors. This is why measurement must consider the whole campaign, not only whether a URL was clicked.

2.3 Human Factors in Susceptibility

Research on phishing susceptibility draws on dual-process theories of judgement. Kahneman distinguishes fast, automatic processing from slow, deliberate reasoning [12]. Most email is processed quickly: the reader classifies the message by sender, subject and visual familiarity, and acts on a heuristic rather than inspecting the underlying link. Cialdini's principles of

influence — authority, scarcity, urgency, social proof, liking and commitment — describe the levers that a lure pulls [13]. A message from 'IT Security' that threatens account deletion within 24 hours combines authority, scarcity and loss aversion in a single sentence.

Downs et al. found that users with more web experience relied on a wider range of cues, including URL structure and security indicators, whereas less experienced users relied on surface features such as logos and sender names [14]. Jagatic et al. showed that a message that appeared to come from a friend was far more effective than the same message from a stranger [15]. Vishwanath et al. integrated individual differences, information processing and involvement into a model in which heuristic processing and lack of systematic scrutiny increased vulnerability [16]. Together these studies suggest that susceptibility is not a stable personality trait but a context-dependent outcome of cue availability, cognitive load and prior experience.

Organisational context adds further variation. Sales and finance staff handle external requests, invoices and payment approvals as part of their daily work, so a convincing business message is congruent with their task. IT staff encounter security warnings constantly and may be either more sceptical or more habituated. Junior employees may be reluctant to question an executive instruction; senior employees may be less accustomed to scrutiny. Workload, fatigue, device, time of day and the presence of a reporting culture all influence the outcome. A measurement platform should therefore report results by department and by lure, not only as a single organisational average.

2.4 Awareness Interventions

Awareness interventions range from posters and annual e-learning to embedded training, serious games and simulated phishing with immediate feedback. Kumaraguru et al. evaluated an embedded training email that taught users at the moment they encountered a phishing message; participants improved their detection performance relative to a control group [17], [18]. A later longitudinal study found that repeated training and feedback were needed to sustain the effect, and that retention declined over time [19]. Caputo et al. explored embedded training in a spear-phishing exercise and highlighted the tension between realistic deception and the ethical obligation to inform participants [20].

Wash and Cooper examined what makes training credible. Employees valued stories from people like themselves and concrete examples over abstract warnings; generic facts and fear appeals were less persuasive [21]. This finding supports a design in which the coaching page uses the exact lure that the participant has just encountered, explains the specific red flags in that message, and avoids blaming the individual. The teachable moment is strongest immediately after the action, when the consequence is concrete and the message is still on screen.

Simulation-based training has become the dominant industry model. A campaign is sent, behaviour is recorded, and users who fail are enrolled in training. The approach has obvious advantages: it measures behaviour rather than self-reported knowledge, and it can be repeated. It also has known weaknesses. Click rates are sensitive to lure quality, audience and timing; repeated simulations can breed habituation or resentment; and a focus on catching individuals can discourage reporting. The design in this project therefore treats reporting as a first-class positive behaviour and couples every failure with immediate, non-punitive coaching.

2.5 Simulation Platforms and Measurement

A phishing simulation platform must perform five functions: author a credible lure, deliver it to a defined audience, track per-recipient interaction, present a landing page, and report the results. GoPhish provides these functions in a single Go binary with a web console, campaign scheduling, templates and landing pages [9]. King Phisher offers a client/server architecture, campaign management and a template library [10]. Both are mature and widely used, but neither includes a built-in training curriculum, a risk score or an integrated printable report; those functions are left to separate tools or manual analysis.

Commercial platforms such as KnowBe4, Proofpoint and Cofense add large content libraries, automated enrolment, benchmarking and reporting. They reduce operational effort but introduce recurring cost, data-processing agreements and a dependency on the vendor's definition of risk. For a college or a small enterprise, the total cost of ownership and the difficulty of explaining the metric to employees can outweigh the convenience. The platform described in this report is deliberately narrower: it implements the core measurement loop, keeps the data local, and exposes the formula behind every number.

Measurement itself is contested. Open rates are unreliable because mail clients block tracking pixels or proxy images. Click rates depend on whether a click on a harmless landing page is counted as a failure, and submission rates depend on whether the landing page actually requests credentials. Reporting rates depend on how easy it is to report and whether reporters are praised or punished. Time-to-click and repeat-click measures add useful context. A credible report should define each metric operationally, state the denominator, and avoid comparing campaigns that differ in audience or difficulty. Table 2.2 compares representative platforms and studies.

Approach	Primary focus	Strengths	Limitations
GoPhish	Campaign delivery	Lightweight, open source, templates, landing pages	No training loop or risk model
King Phisher	Campaign delivery	Client/server, scheduling, extensible	Operational setup; analytics external
KnowBe4 / Proofpoint	Managed awareness	Content library, automation, benchmarking	Cost, vendor data processing, opaque scoring
Kumaraguru et al.	Embedded training	Teachable moment, empirical evaluation	Email client dependency; retention decay
Caputo et al.	Spear-phishing exercise	Realistic target group; ethics discussion	Small scale; limited platform reuse
This work	Measurement and training loop	Self-hosted, private, integrated, reproducible	Synthetic pilot; no SMTP; small cohort

Table 2.2: *Comparison of representative platforms and studies*

2.6 Technical Controls and the Human Layer

Email authentication is the first line of defence. SPF authorises sending hosts, DKIM cryptographically signs messages, and DMARC tells the receiver what to do when alignment fails. Together they make exact-domain spoofing harder, but they do not prevent look-alike domains, compromised legitimate accounts or messages sent through a permitted bulk provider [7]. Link rewriting and detonation sandboxes can block known malicious URLs, but attackers rotate domains and abuse trusted file-sharing services. Multi-factor authentication raises the cost of credential theft, yet push-notification fatigue and adversary-in-the-middle proxies can defeat one-time codes [22], [8]. Phishing-resistant authenticators such as FIDO2 security keys address the proxy problem, but deployment is uneven.

The implication is not that technical controls are futile, but that they shift the residual risk to the person. A defence-in-depth strategy combines prevention, detection and response: filtering and authentication reduce volume; endpoint and identity controls limit the consequences of a click; and a fast, blameless reporting channel turns employees into sensors. The platform in this

project is positioned at the measurement and response end of that chain. It does not replace a mail gateway or an endpoint agent; it provides the evidence needed to target them.

2.7 Ethics and Legal Considerations

Phishing simulations involve deception, even when the deception is benign. The ethical literature converges on several safeguards: informed consent at the programme level, a clear statement of purpose, an opt-out for vulnerable staff, immediate debriefing after the exercise, no collection of real secrets, and a commitment not to use individual results for discipline [20], [23]. The General Data Protection Regulation in the European Union and the Digital Personal Data Protection Act in India impose additional duties of purpose limitation, minimisation and security [24], [25]. Employee monitoring law and works-council agreements may apply in some jurisdictions.

The platform addresses these constraints by design rather than by policy alone. It never sends real email, so no external recipient can be deceived. It never stores the values entered into a landing page; it records only which field names were submitted. It uses fictional employees and a fictional company in the demonstration dataset. It displays a persistent banner stating that the environment is a simulation. It gives the participant immediate feedback and a route to report the message. These choices make the artefact suitable for a college laboratory and provide a template for a consented field deployment.

2.8 Research Gap

The literature establishes that human susceptibility is context-dependent, that embedded feedback is more effective than annual awareness, and that simulation is a useful measurement method. It also reveals three gaps. First, most open-source tools stop at campaign delivery; training, risk scoring and reporting are separate. Second, most studies evaluate a training intervention rather than the measurement instrument, so the reproducibility of the analytics is rarely discussed. Third, small organisations lack a deployable artefact that respects privacy by design and can be operated without a cloud subscription.

This project addresses the gap by building an integrated, self-hosted platform and by evaluating it on a transparent synthetic cohort. The contribution is not a new theory of phishing; it is an engineering artefact and an evaluation method that make the theory

operational for a small team. The next chapter turns the requirements implied by this review into a system design.

2.9 Summary

Phishing exploits human judgement through authority, urgency and familiarity. Technical controls reduce volume and consequence but leave a residual human decision. Awareness training works best when it is concrete, timely and repeated, and simulation is the most direct way to measure behaviour. Existing platforms are either commercial and opaque or open and incomplete. The gap is an integrated, privacy-preserving, reproducible platform that connects campaign delivery, analytics, coaching and reporting. Chapters 3 to 8 describe how that platform was designed, built and tested.

Chapter 3: Proposed System

This chapter specifies the proposed system. It states the design goals, presents the architecture and the data model, derives the functional and non-functional requirements, describes the principal processes and use cases, and analyses the threats that the platform must resist. The design is intentionally modest in its dependencies and aggressive in its auditability: every number shown to an administrator should be traceable to an event in the database.

3.1 System Overview

PhishStudy is a server-rendered web application with two audiences. The **administrator** signs in to a console, creates lures, defines an audience, launches a campaign, monitors the funnel and reads the analytics. The **participant** receives a simulated message in a web inbox, opens it, follows a tokenised link, sees a landing page, and either submits the form, reports the message or leaves. A submission leads to a coaching page that explains the red flags in that exact message and offers an eight-item quiz. The administrator can export the results, print a report and review the audit log.

The system is organised into seven modules: identity and access, campaign management, lure management, people and consent, participant experience, analytics and training, and reporting. A small routing layer maps HTTP requests to views, and a persistence layer wraps SQLite. There is no JavaScript framework, no external CSS, no third-party analytics and no outbound network call at runtime. This makes the system easy to audit and easy to run on a laptop or a departmental server.

3.2 Design Goals

The design is guided by eight goals, derived from the literature review and the constraints of a college project:

- **Safety before realism.** No real email, no real credentials and no external target. Realism is achieved through the quality of the simulated message, not through live delivery.
- **Measurability.** Every participant action is stored as a timestamped event with a well-defined denominator, so that rates can be recomputed and audited.

- **Privacy by design.** The minimum data needed for the metric is collected. Submitted values are discarded; only field names are retained.
- **Immediate feedback.** A failure is followed by an explanation and a short assessment, not by a punitive email to a manager.
- **Operational simplicity.** A single Python process, a single SQLite file and no external services. A new administrator can be productive in minutes.
- **Auditability.** Authentication events, campaign changes and exports are written to an append-only audit log with actor, action, detail and timestamp.
- **Extensibility.** Lures, audiences, quiz questions and metrics are data, not hard-coded branches, so the platform can grow without a rewrite.
- **Accessibility and clarity.** Semantic HTML, readable contrast, large touch targets and a layout that works on a phone, because participants may open the message on a mobile device.

3.3 System Architecture

Figure 3.1 shows the layered architecture. The presentation layer renders HTML templates and a small design system. The application layer contains the HTTP server, routing, authentication, CSRF checks and request handlers. The domain layer contains campaign lifecycle rules, analytics functions, risk scoring and quiz logic. The persistence layer is SQLite in WAL mode with foreign keys enabled. A cross-cutting security module provides password hashing, session tokens, CSRF tokens and login throttling.

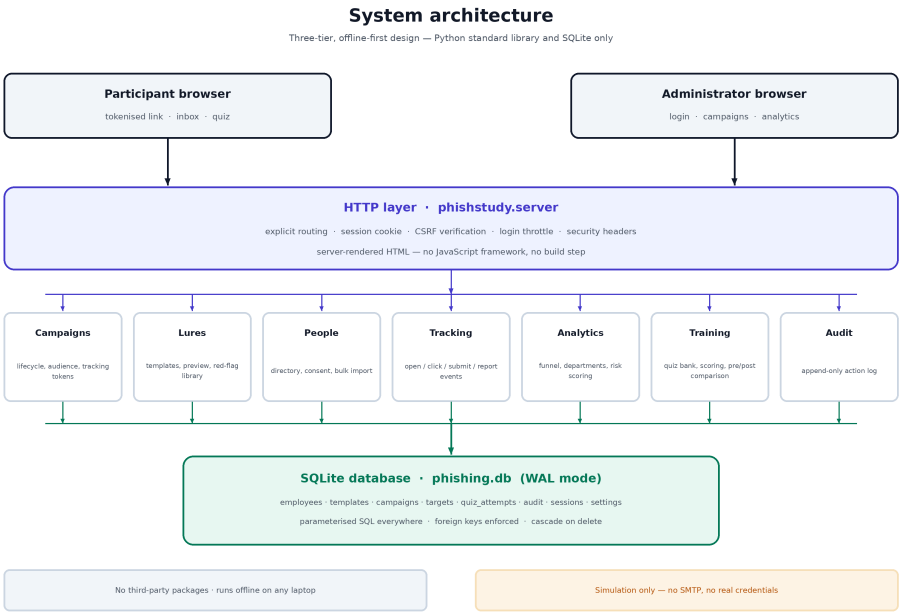


Figure 3.1: *Layered architecture of PhishStudy. Every module reaches the database through the persistence layer, and every request passes through the security module.*

The request lifecycle is deliberately simple. The server parses the request, opens a short-lived database connection, resolves the session cookie, enforces the public/protected route boundary, validates the CSRF token for state-changing requests, dispatches to a view, renders HTML and closes the connection. There is no ORM and no connection pool. SQLite's write-ahead log allows concurrent readers while a campaign is being updated, which is sufficient for the expected load of a departmental deployment.

3.4 Functional Requirements

The functional requirements were derived from the campaign lifecycle and from the questions an administrator must answer. They are numbered for traceability to the tests in Chapter 8. Table 3.1 lists the requirements and their priority; 'M' denotes mandatory for the present release and 'S' denotes should-have.

ID	Requirement	Priority
FR-01	Authenticate administrators with a username and password; lock out repeated failures.	M
FR-02	Create, edit, duplicate, activate and deactivate lure templates.	M
FR-03	Define an audience by department and preview the recipient count.	M
FR-04	Create a campaign in Draft and move it through Scheduled, Active, Paused, Completed and Archived.	M
FR-05	Generate a unique, unguessable tracking token for every recipient.	M
FR-06	Render a simulated inbox and a faithful HTML mail view for each token.	M
FR-07	Record open, click, submit and report events with timestamps.	M
FR-08	Discard submitted values and retain only the field names.	M

FR-09	Show a coaching page with the red flags of the exact lure after submission.	M
FR-10	Provide an eight-item quiz with scoring, feedback and attempt history.	M
FR-11	Compute funnel, campaign, department, lure and difficulty statistics.	M
FR-12	Compute a bounded individual risk score and band.	M
FR-13	Export the results as CSV and JSON and render a printable report.	M
FR-14	Maintain an audit log of authentication, campaign and export events.	M
FR-15	Allow an administrator to reset the demonstration dataset.	S
FR-16	Show a persistent simulation banner on every page.	M
FR-17	Support a participant reporting flow with a confirmation page.	M
FR-18	Provide a settings page for organisation name, domain and pass mark.	S

Table 3.1: *Functional requirements*

3.5 Non-Functional Requirements

Non-functional requirements constrain how the system behaves. They are especially important for a security product, because a weak administrative console would undermine the credibility of the measurement. Table 3.2 lists the requirements and the design response.

ID	Quality attribute	Requirement	Design response
NFR-01	Security	Resist credential stuffing, session theft, CSRF and XSS.	PBKDF2-SHA256, hashed session tokens, per-session CSRF, output escaping, CSP.
NFR-02	Privacy	Collect the minimum behavioural data.	No value capture, no third-party scripts, aggregate exports.
NFR-03	Performance	Interactive pages under 200 ms at departmental scale.	Indexed queries, WAL, server-side aggregation; measured in Chapter 8.
NFR-04	Portability	Run on Linux, macOS or Windows with Python 3.11+.	Standard library only; SQLite file; no native extensions.
NFR-05	Reliability	Survive restarts without losing committed events.	WAL journal, foreign keys, explicit commits, idempotent timestamps.
NFR-06	Usability	A new administrator can launch a campaign without training.	Guided forms, inline help, empty states, consistent navigation.
NFR-07	Accessibility	Readable on mobile and with keyboard navigation.	Semantic HTML, focus styles, contrast, responsive layout.
NFR-08	Maintainability	Small, testable modules with no hidden coupling.	Package split by concern; 24 automated tests; typed helpers.
NFR-09	Auditability	Reconstruct who changed what and when.	Append-only audit table; timestamps in local time; actor field.
NFR-10	Scalability	Support at least 1,000 employees and 5,000 targets.	Set-based SQL, indexes, pagination; benchmark in Chapter 8.

Table 3.2: *Non-functional requirements and design response*

3.6 Data Model

The data model has nine tables. **admins** stores the console account and its password hash. **sessions** stores a hash of each session token, the CSRF secret, a flash message and an expiry time. **employees** stores the fictional population, department, designation, active flag and consent flag. **templates** stores the lure: sender, subject, body, landing form, difficulty, category and red flags. **campaigns** stores the lifecycle state, phase, audience, schedule and timestamps.

targets is the join between a campaign and an employee and carries the unique token and the four behavioural timestamps. **quiz_attempts** stores the score, total, chosen options and time. **audit** stores actor, action, detail and time. **settings** stores organisation-level configuration.

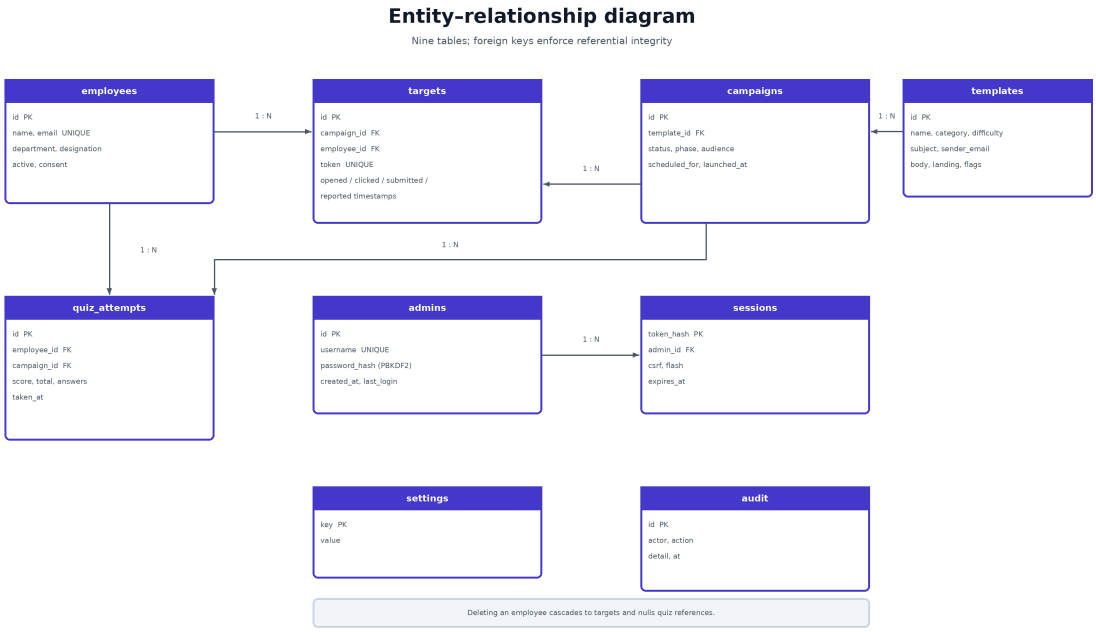


Figure 3.2: *Entity–relationship diagram. Foreign keys enforce referential integrity; deleting an employee cascades to targets and nulls quiz references.*

Two design choices deserve explanation. First, the behavioural events are nullable timestamps on the target row rather than separate event rows. This makes the funnel query a set of `COUNT` expressions over one table, which is fast and easy to audit. The trade-off is that the model cannot represent repeated clicks on the same target; for the present scope, first-event semantics are sufficient and are stated in the metric definitions. Second, the submitted field names are stored as a JSON array, while the values are never written. This preserves the ability to say 'the password field was filled' without retaining a secret.

3.7 Process Design

Figure 3.3 is a level-0 data-flow diagram. The participant exchanges simulated mail and event notifications with the campaign engine. The administrator manages campaigns and audiences. The campaign engine writes target events, the analytics process reads and aggregates them, and the training process stores quiz attempts and returns coaching. The three data stores correspond to the target events, the campaign definitions and the quiz history.

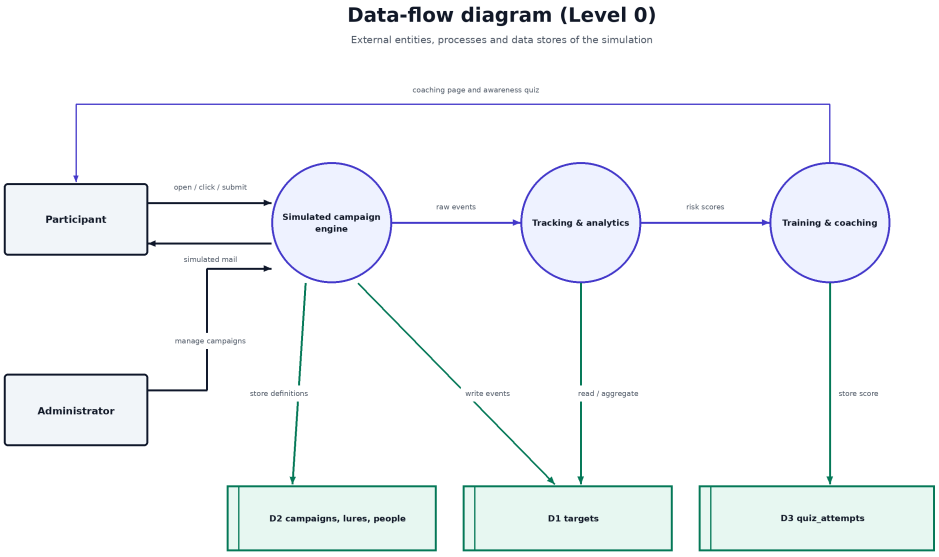


Figure 3.3: *Level-0 data-flow diagram. Events flow from the participant to the campaign engine and thence to analytics and training.*

The campaign lifecycle is a small state machine. A campaign begins in **Draft**, moves to **Scheduled** when a launch time is set, becomes **Active** when launched, may be **Paused** and resumed, and ends in **Completed** or **Archived**. A scheduled campaign materialises its targets at launch, which freezes the audience and guarantees that the denominator cannot change after the first event. Pausing stops new sends but leaves existing tokens valid, so a participant who opens an old message still sees a coherent page. These rules are enforced in the service layer, not only in the user interface.

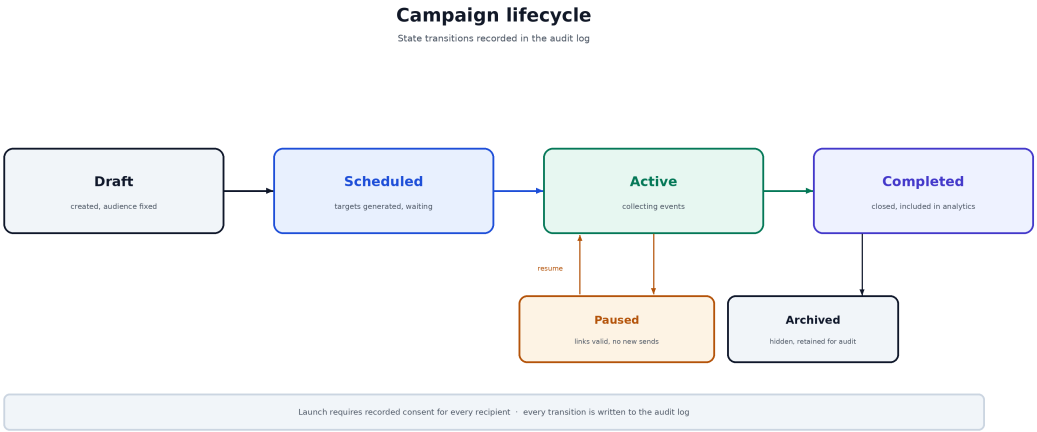


Figure 3.4: *Campaign lifecycle. Targets are materialised at launch so that the denominator is stable for the whole campaign.*

3.8 Use Cases

Figure 3.5 shows the actors and the principal use cases. The administrator manages lures, people, campaigns, analytics, training and settings. The participant opens a message, clicks, submits, reports, reads coaching and takes the quiz. The system actor performs scheduled launches, token generation and aggregation.

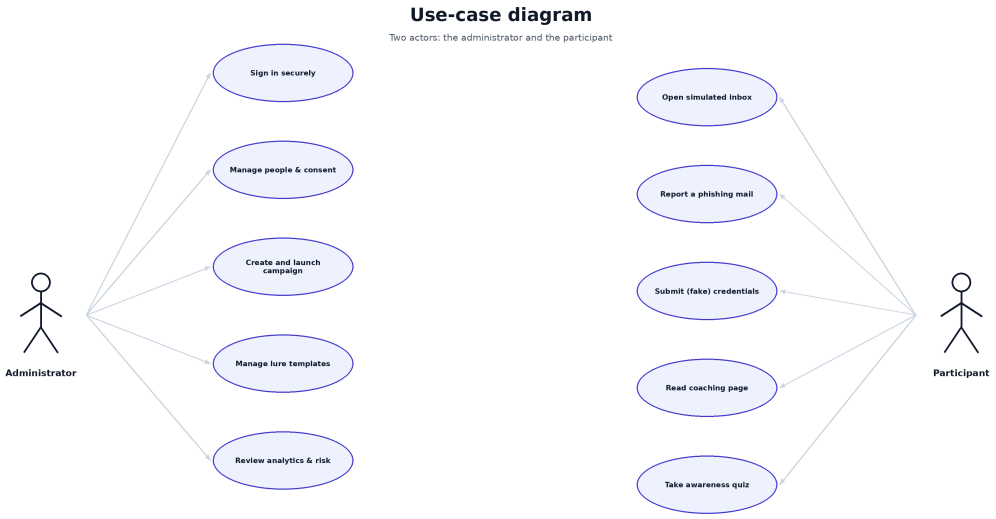


Figure 3.5: *Use-case diagram for the administrator, the participant and the scheduled system actor.*

Three use cases are central to the contribution and are described below.

3.8.1 Launch a campaign

Precondition: the administrator is authenticated and at least one active lure and one employee exist. **Main flow:** the administrator selects a lure, names the campaign, chooses a phase and audience, reviews the recipient count, and saves. The campaign is created in Draft. On launch, the system materialises one target per recipient with a random token, records the launch time, changes the status to Active and writes an audit entry. **Postcondition:** every recipient has exactly one token and the denominator is fixed. **Exception:** an empty audience is rejected with a validation message.

3.8.2 Respond to a simulated message

Precondition: the participant holds a valid token. **Main flow:** the participant opens the mail view, which records the first open. A click records the first click and renders the landing page. A

submission records the first submit, discards the values, stores the field names and redirects to coaching. A report records the first report and shows a confirmation. **Postcondition:** the event is timestamped once; repeated requests are idempotent. **Exception:** an unknown token returns a neutral 404 page without revealing whether the token ever existed.

3.8.3 Analyse and export results

Precondition: the administrator is authenticated. **Main flow:** the analytics view aggregates targets by campaign, department, difficulty and template; the risk view ranks employees; the report view renders a printable summary; the export endpoints stream CSV and JSON. **Postcondition:** the numbers on screen and in the export are produced by the same analytics functions, so they cannot drift. **Exception:** an export request without a session is redirected to the login page.

3.9 Threat Model and Safeguards

The platform is itself a security-sensitive application: it holds behavioural data and an administrative session that can launch campaigns. A lightweight STRIDE analysis was used to identify threats and to choose proportionate controls (Table 3.3). The analysis assumes a network attacker, a malicious participant and an authorised administrator who makes a mistake; it does not assume a compromised host or a compromised Python runtime.

Threat	Example	Safeguard
Spoofing	Credential stuffing against the admin login	PBKDF2-SHA256 with 260,000 iterations; login throttle; generic error message
Tampering	Forged campaign status or target timestamps	Server-side state machine; parameterised SQL; CSRF token on every state change
Repudiation	An administrator denies launching a campaign	Append-only audit log with actor, action, detail and timestamp
Information disclosure	Session theft or export leakage	HttpOnly, SameSite=Lax cookie; hashed session tokens; no values stored; access-controlled exports
Denial of service	Repeated login or quiz submissions	Per-IP and per-account throttle; bounded request body; short transactions
Elevation of privilege	Participant reaches admin routes	Central protected-route check; no role parameter in the session; 404 on unknown tokens

Table 3.3: *STRIDE threat model and safeguards*

3.10 Design Trade-offs and Summary

Every design choice excludes alternatives. Using the standard library instead of a web framework keeps the dependency surface at zero, but it means writing routing, sessions and CSRF handling by hand; the tests in Chapter 8 are the compensation. Server-rendered HTML

instead of a single-page application keeps the participant experience fast and accessible, but it limits rich interactivity. SQLite instead of PostgreSQL makes deployment trivial, but it constrains multi-writer concurrency. A synthetic cohort instead of real employees makes the study ethical and reproducible, but it limits external validity. These trade-offs are stated here so that the evaluation in Chapters 8 and 9 can be read against the intended design rather than against an unstated ideal.

In summary, the proposed system is a layered, dependency-free web application that connects campaign delivery, behavioural telemetry, analytics, coaching and reporting. Its requirements are traceable, its data model is normalised, its lifecycle is explicit, and its threat model is matched by concrete controls. The next chapter explains the methodology used to evaluate it and the simulation model that generates the synthetic cohort.

Chapter 4: Methodology

This chapter explains how the system was designed and evaluated. It describes the design-science stance, the simulation model that produces the synthetic cohort, the operational definitions of every metric, the risk-scoring formula, the training design, the data-protection rules and the validation strategy. The chapter is deliberately explicit about what the synthetic evaluation can and cannot establish, because that distinction governs how the results in Chapter 9 should be read.

4.1 Research Design

The project follows the design-science research paradigm described by Hevner et al., in which knowledge is produced through the construction and evaluation of an artefact [26]. The artefact is the PhishStudy platform: a software system, a data model and an evaluation pipeline. The evaluation has two strands. The first is **technical validation**: does the system behave correctly under unit, integration, security and load tests? The second is **behavioural illustration**: does the analytics pipeline recover the patterns that were designed into a calibrated synthetic cohort, and are those patterns presented in a way that a security manager can act on?

The two strands answer different questions. Technical validation can establish that the software is correct with respect to its specification. Behavioural illustration cannot establish that a real workforce would behave in the same way; it can only show that the instrument is capable of measuring and explaining a realistic pattern. The report keeps these claims separate. The field deployment that would support causal claims about training is planned for Semester VIII and is described in Chapter 6.

The development process was iterative. Requirements were elicited from the literature and from the practical constraints of a college laboratory; a prototype was built; each prototype was exercised through the participant and administrator flows; defects were recorded and fixed; and the test suite was expanded after each defect. This is closer to action research than to a waterfall, and it produced a system that is shaped by use rather than by an abstract specification.

4.2 Simulation Model

The demonstration dataset represents a fictional company, NorthStar Technologies, with 24 employees in six departments. Each employee has a **latent susceptibility** value between zero and one. The values are hand-tuned so that the cohort has a believable spread: IT has the lowest values, Sales and Finance the highest, and the other departments sit in between. The latent value is not shown to the administrator; it is an input to the data generator, used to make the same people behave coherently across campaigns.

Each campaign has three **prevalence targets**: the percentage of recipients expected to click, to submit and to report. At launch, recipients are ranked by latent susceptibility plus a small Gaussian noise term. The top-ranked recipients become clickers, the highest-ranked become submitters, and reporters are drawn from the non-clickers with the lowest latent values. This quota-based construction guarantees that the cohort exhibits the intended aggregate pattern while preserving individual coherence: a person who is susceptible in one campaign is likely to be susceptible in the next. Open events occur with probability 0.92, and reporters always open the message. Timestamps are drawn uniformly within realistic windows: 0.4–7 hours to open, 0.2–5 hours from open to click, 0.1–1.5 hours from click to submission, and 0.3–20 hours from launch to report.

The generator is seeded with a fixed value, so every fresh installation produces the same numbers. This is a deliberate methodological choice. It makes the report reproducible, allows a reviewer to recompute every percentage, and ensures that screenshots and tables agree. It also means that the results are a **calibration scenario**, not a random sample. The five campaign targets were chosen to represent a plausible baseline, a training intervention and a live campaign; they were not estimated from a real organisation. The evaluation therefore tests the measurement pipeline against known inputs, not the effectiveness of training in the field.

The lure difficulty label is metadata attached to a template. It is **not** used as a causal input to the outcome model. In the seeded schedule, difficulty is correlated with campaign and phase, so the difficulty table in Chapter 9 must not be read as an experimental comparison of easy, medium and hard lures. This limitation is stated again in the threats-to-validity section. A future version of the generator will randomise difficulty within phase so that the effect can be isolated.

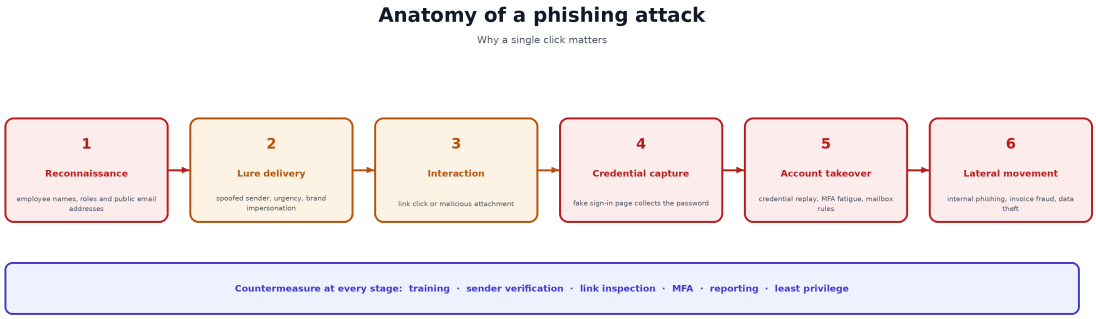


Figure 4.1: *Anatomy of a phishing attack, adapted from the campaign phases modelled by the platform. The platform instruments planning, delivery, interaction, capture and feedback; it stops short of exploitation.*

4.3 Campaign Lifecycle and Experimental Procedure

The experimental procedure follows the lifecycle described in Chapter 3. In the baseline phase, three campaigns are launched one week apart to the whole population: a password-expiry lure (easy), a festival-bonus lure (medium) and an M365 security alert (hard). Each campaign runs for seven days. The first two establish a broad baseline; the third provides the like-for-like stimulus for the post-training comparison.

After the third campaign, every employee takes the eight-item quiz as a pre-training measure. The coaching page and quiz are available to all participants, and the administrator can see aggregate scores but not individual answers in the dashboard. A post-training campaign then repeats the M365 lure to the whole population. Because the lure, audience and difficulty are held constant, any change in the click, submission or report rate can be attributed to the passage of time and the training, subject to the limitations discussed in Chapter 9. A fifth campaign, a courier-delivery lure, is launched to the Operations and Sales departments only, to demonstrate a live, smaller-audience campaign.

All campaigns are launched manually by the administrator in the demonstration. The scheduler is implemented, but the seed uses explicit launch times so that the dataset is reproducible. The campaign state is advanced through the console, and every transition is written to the audit log. The procedure is summarised in Table 5.2.

4.4 Metrics and Operational Definitions

A metric is only useful if its denominator and its event semantics are explicit. Table 4.1 defines the metrics used in the dashboard, the report and the exports. All rates are percentages of delivered targets, not of employees, because an employee may receive several campaigns. A

target is counted once per event, using the first timestamp; repeated requests do not inflate the numerator.

Metric	Definition	Denominator	Interpretation
Delivered	Targets materialised at launch	—	Size of the campaign
Open rate	Targets with opened_at set	Delivered	Message was rendered; pixel blocking can undercount
Click rate	Targets with clicked_at set	Delivered	Link was followed; strongest behavioural signal
Submission rate	Targets with submitted_at set	Delivered	Form was submitted; values are not stored
Report rate	Targets with reported_at set	Delivered	Positive security behaviour
Median time to click	Median of clicked_at − sent_at	Clickers	Speed of engagement, in hours
Repeat click rate	Share of employees clicking ≥2 campaigns	Employees	Persistence of susceptibility
Quiz score	Correct answers out of eight	Attempts	Knowledge after coaching
Pass rate	Attempts scoring ≥6/8	Attempts	Threshold used by the training module
Risk score	Composite formula in Section 4.5	Employee	Bounded 0–100; higher is riskier

Table 4.1: *Operational definitions of the reported metrics*

Two metrics require a caveat. Open rate is measured by a tracking pixel; privacy proxies and image blocking can suppress the request, so a low open rate is not necessarily a low level of attention. Report rate is modelled as an alternative to engagement in the synthetic cohort: reporters are drawn from non-clickers. In a real deployment a person could click and then report, so the reporting and clicking categories would overlap. The platform records both timestamps independently and can represent that overlap; the synthetic generator chooses not to, for narrative clarity.

4.5 Composite Risk Score

The individual risk score combines four observable signals. Submission is weighted most heavily because it represents the greatest potential harm; clicking is weighted next; reporting is treated as a protective behaviour and subtracts a small amount; and quiz performance contributes a penalty when knowledge is weak. The formula is:

Risk formula risk = 0.60 × submission_rate + 0.25 × click_rate + 15 × (1 − quiz_accuracy) − 0.10 × report_rate

The raw score is clipped to the range 0–100 and rounded to one decimal place. When an employee has no quiz attempt, the training penalty is set to 10, a neutral value that does not reward or punish missing data. The bands are shown in Table 4.2. The weights are a design choice, not an empirical finding; they were chosen so that a person who submits on every

campaign and never reports is classified as critical, while a person who clicks but never submits, reports consistently and scores well on the quiz remains in the low or moderate band.

Band	Score range	Interpretation	Recommended action
Low	0 – 24.9	Resilient behaviour; reports or ignores most lures	Maintain awareness; no individual action
Moderate	25 – 49.9	Occasional clicks; limited submission	Targeted micro-training; monitor trend
High	50 – 74.9	Repeated clicks or a submission	Coaching conversation; priority for the next round
Critical	75 – 100	Repeated submissions; no reporting	Immediate support; review account controls; avoid blame

Table 4.2: *Risk bands and recommended action*

The score is deliberately transparent. An administrator can see the four inputs, and the report explains the formula. This is a contrast to commercial platforms that present a single 'phishing-prone' percentage without a published derivation. The trade-off is that a simple linear formula cannot capture every interaction; it is a communication and triage tool, not a psychometric instrument.

4.6 Training Design

The training is embedded in the moment of failure. When a participant submits a landing page, the platform does not simply display a generic warning. It renders the original message alongside a list of the specific red flags in that message: the look-alike domain, the urgency cue, the generic greeting, the request for a password or OTP. It then offers an eight-item quiz that tests recognition, safe action, MFA, password practice and incident response. The quiz is scored immediately, with the correct answer and a short explanation for each item.

The item bank is fixed so that scores are comparable across rounds. The pass mark is six out of eight (75%), stored in the settings table. The quiz is not designed to certify competence; it is designed to reinforce the coaching and to give the administrator a second, knowledge-based signal alongside behaviour. The questions avoid jargon and use the same fictional brand and domain as the lures, so that the transfer from the simulation to the assessment is direct. Appendix B reproduces the instrument.

4.7 Data Collection, Consent and Privacy

The platform collects four categories of data: identity data for the fictional employees (name, email, department, designation), campaign definitions, behavioural timestamps and quiz

responses. It does not collect passwords, OTPs, payment details or any other value entered into a landing page. The submission handler reads the field names from the form, stores them as a JSON array, and discards the values before rendering the response. This is the most important privacy control in the system: even if the database were disclosed, it would not contain a credential.

In a real deployment, the lawful basis for processing would be the employer's legitimate interest in information security, supported by a transparency notice, a programme-level consent process and a data-protection impact assessment. The platform provides the technical preconditions: purpose limitation (behavioural data is used only for the awareness programme), minimisation (no values), storage limitation (a retention setting is planned), integrity (hashed tokens, parameterised queries) and accountability (audit log). The synthetic demonstration avoids the legal question entirely, but the design is intended to be deployable under the GDPR and the Indian DPDP Act with the appropriate organisational safeguards [24], [25].

4.8 Validation Approach

Validation combines four levels. **Unit tests** exercise the security primitives and the analytics functions in isolation. **Integration tests** drive the HTTP server through login, campaign creation, participant flows, exports and quiz submission. **Security tests** probe authentication, CSRF, session handling, open redirects and output escaping. **Data-integrity tests** check that the funnel is monotonic, that no submission exists without a click, that department totals reconcile with the overview, and that the seeded dataset has the expected shape. A performance test measures response times at the demonstration scale and at a synthetically enlarged scale. A heuristic usability review applies Nielsen's ten heuristics to the principal screens [27].

The validation is not a substitute for a field trial. It can show that the software does what it claims and that the analytics are internally consistent; it cannot show that the metrics predict real-world compromise. The next chapter describes the experimental setup in enough detail for another team to reproduce the synthetic study.

4.9 Ethical Framework

The project follows five ethical principles. **No real deception:** no message leaves the application, and the demonstration population is fictional. **No real secrets:** submitted values

are discarded. **Informed participation:** in a field deployment, employees would receive a programme notice and an opt-out. **Blameless feedback:** individual results are used for coaching, not discipline, and the report recommends against punitive use. **Proportionate data:** only the minimum data needed for the metric is retained, and access to the console is restricted.

These principles are not merely stated in a policy; they are reflected in the code. The simulation banner is rendered on every page. The participant routes are tokenised and do not expose a directory of employees. The export endpoints require an administrator session. The audit log records exports. The report itself aggregates departments and does not publish individual quiz answers. The ethics review is therefore an input to the design, not an afterthought.

4.10 Summary

The methodology combines design-science construction with a transparent synthetic evaluation. The simulation model is quota-based and calibrated, not predictive; it exists to test the measurement pipeline and to produce a coherent demonstration. The metrics are defined operationally, the risk score is published, the training is embedded, and the privacy controls are enforced in code. The next chapter describes the experimental setup in detail.

Chapter 5: Experimental Setup

This chapter describes the environment, the synthetic population, the campaign schedule, the lure catalogue, the quiz instrument and the procedure used to produce the results in Chapter 9. The level of detail is intentional: a reader should be able to recreate the dataset from the seed and compare their numbers with those reported here.

5.1 Environment

The platform was developed and evaluated on a Linux workstation with an Intel Core i7-13620H processor and 16 GB of RAM. The runtime is Python 3.14.7 with the standard library only; SQLite 3.53.4 provides persistence. LibreOffice 26.8 was used to render the report and to convert it to PDF. Chromium in headless mode was used to capture the screen images in Chapter 7. No container, cloud service or external database was used. The application listens on 127.0.0.1 by default and binds to a configurable host and port.

5.2 Synthetic Organisation

NorthStar Technologies is a fictional 24-person organisation with six departments of four employees each. The names, email addresses and designations are synthetic; the domain `northstar.example` is reserved for documentation and cannot receive mail. The population was designed to include a range of roles that plausibly encounter external requests: account executives and a regional sales lead; accounts, billing, audit and financial analysts; HR managers, recruiters and payroll staff; legal counsel, compliance and contract staff; operations leads, process analysts and support staff; and IT engineers, administrators, analysts and DevOps staff. Table 5.1 summarises the population.

Department	Employees	Representative roles	Expected exposure
IT	4	Systems engineer, network administrator, security analyst, DevOps	Low (security-trained)
HR	4	HR manager, recruiter, payroll officer, HR executive	Moderate
Finance	4	Accounts manager, financial analyst, auditor, billing	High (payments, invoices)
Sales	4	Regional lead, account executive, business development, inside sales	High (external mail, urgency)
Operations	4	Operations lead, process analyst, logistics, support	Moderate
Legal	4	Legal counsel, compliance officer, contract analyst, paralegal	Moderate

Table 5.1: *Composition of the synthetic population*

The latent susceptibility values were assigned by department and then varied within each department to avoid a deterministic mapping. IT has the lowest values, Sales and Finance the highest, and HR, Operations and Legal occupy the middle. The values are an input to the generator, not a claim about real employees. Their purpose is to create a cohort in which individual behaviour is coherent across campaigns, so that repeat-clicking and risk ranking are meaningful.

5.3 Campaign Schedule

Five campaigns were launched between 24 August and 18 October 2026, and a sixth is scheduled for January 2027. The first three form the baseline; the fourth repeats the M365 lure after training; the fifth is a live, smaller-audience campaign. Table 5.2 gives the schedule and the number of recipients.

#	Campaign	Launched	Phase	Lure	Audience	Sent
1	Q1 Baseline — Password Expiry	24 Aug 2026	Baseline	IT Password Expiry	All (24)	24
2	Q1 Baseline — Festival Bonus	31 Aug 2026	Baseline	HR Festival Bonus	All (24)	24
3	Q2 Baseline — M365 Alert	07 Sep 2026	Baseline	M365 Security Alert	All (24)	24
4	Q3 Post-training — M365 Alert	08 Oct 2026	Post-training	M365 Security Alert	All (24)	24
5	Q3 Live — Courier Delivery	18 Oct 2026	Live	Courier Delivery Failure	Sales + Operations (8)	8
6	Q4 Planned — Shared Document	Jan 2027	Planned	Shared Document Review	All (24)	0

Table 5.2: *Campaign schedule*

The total number of delivered targets is 104. The sixth campaign is in Draft/Scheduled state and is excluded from all rate calculations because it has no delivered targets. This is why the overview reports five campaigns rather than six: the denominator is delivered targets, not campaign records.

5.4 Lure Catalogue

Six lure templates were authored. Each has a sender name, a look-alike domain, a subject, an HTML body, a landing form and a list of red flags used by the coaching page. Table 5.3 summarises the catalogue. The templates cover credential theft, financial reward, malware delivery, business email compromise and document sharing, which are the categories most frequently encountered in the literature.

Lure	Category	Difficulty	Sender domain	Landing fields	Red flags
IT Password Expiry	Credential theft	Easy	northstar-supportdesk.com		4

				Email, password, OTP	
HR Festival Bonus	Financial lure	Medium	northstar-hrportal.com	Name, employee ID / DOB	4
M365 Security Alert	Credential theft	Hard	micorsoft-secure-auth.com	Email, password	4
Courier Delivery Failure	Malware delivery	Medium	swiftcourier-delivery.net	Tracking reference	4
CEO Gift Card Request	Business email compromise	Hard	northstar-ceo.example	Reply by email	4
Shared Document Review	Credential theft	Hard	northstar-sharepoint.example	Email, password	4

Table 5.3: *Lure catalogue*

The first four lures were used in launched campaigns; the last two are available in the catalogue but were not delivered in the present study. The CEO gift-card lure is included because BEC is a distinct and high-impact category, and the shared-document lure is included because it exercises a different pretext (collaboration) from the password-expiry and security-alert lures. Both are available for the field pilot.

5.5 Quiz Instrument

The quiz has eight multiple-choice items, each with three options and one correct answer. It covers recognition of a phishing message, domain verification, link inspection, the correct reporting action, the scope of MFA, business email compromise, password practice and post-incident response. The blueprint is shown in Table 5.4 and the full instrument is reproduced in Appendix B. The pass mark is six out of eight.

Item	Theme	What it tests
Q1	Recognition	Urgency combined with an unrecognised sender domain
Q2	Domain verification	A misspelled look-alike domain
Q3	Link inspection	Visible text versus the actual destination
Q4	Reporting	The correct action on a suspected message
Q5	MFA	What multi-factor authentication does and does not protect
Q6	BEC	An executive request for gift cards and secrecy
Q7	Passwords	Unique passphrases and a password manager
Q8	Response	What to do after clicking and entering a password

Table 5.4: *Quiz blueprint*

The items are deliberately practical. They do not ask the participant to define phishing; they ask what to do when a domain is misspelled, when a link is shortened, or when an executive asks for secrecy. The distractors are plausible rather than absurd, so a guess is not rewarded. The same instrument is used before and after training, which permits a paired comparison at the cohort level while acknowledging the practice effect discussed in Chapter 9.

5.6 Experimental Procedure

The procedure has six steps. First, the database is reset to the seeded dataset, which creates the 24 employees, six lures and six campaign records. Second, the three baseline campaigns are launched at weekly intervals and allowed to run for seven days. Third, the participant inbox is used to exercise the flows: open, click, submit and report. Fourth, the pre-training quiz is administered to the cohort. Fifth, the post-training M365 campaign and the live courier campaign are launched. Sixth, the analytics are exported to JSON and the figures and tables in this report are generated from that export.

In the demonstration, the participant actions are generated by the seed rather than typed by 24 people. The administrator can still exercise the live flows manually: the inbox, the mail view, the landing page, the coaching page and the quiz are fully interactive. This dual mode is useful for a demonstration and for testing. The seeded data provides the aggregate cohort; the manual flow proves that a real participant would experience the same pages.

5.7 Variables and Operationalisation

The independent variables in the synthetic design are the campaign (and therefore the lure, difficulty and phase), the department, and the training round. The dependent variables are the click, submission and report indicators, the time to first click, the quiz score and the composite risk score. Employee identity is a grouping variable that links observations across campaigns. The unit of analysis for rates is the target; for repeat behaviour and risk it is the employee.

Because the data are synthetic, no confounding adjustment is required for the generator to produce a coherent dataset. In a field study the same variables would require a pre-registered analysis plan, a power calculation and adjustment for department, role, tenure and baseline score. Chapter 6 describes how the field version will address these issues.

5.8 Hardware and Software

Table 5.5 records the environment so that the performance measurements in Chapter 8 can be interpreted in context.

Component	Specification
Processor	13th Gen Intel Core i7-13620H, 16 logical CPUs
Memory	16 GB
Operating system	Linux 7.2.5

Runtime	Python 3.14.7 (standard library only)
Database	SQLite 3.53.4, WAL mode, foreign keys enabled
Browser	Chromium (headless) for screen capture
Document tooling	LibreOffice 26.8, python-docx, docxcompose, matplotlib
Deployment	Single process, 127.0.0.1:8000 by default

Table 5.5: *Hardware and software environment*

5.9 Summary

The experimental setup is a 24-person synthetic organisation, six lures, five launched campaigns and 104 deliveries, executed on a single Linux workstation with a standard-library Python stack. The next chapter describes the plan that produced this setup and the plan for extending it to a consented field pilot.

Chapter 6: Implementation Plan

This chapter describes how the work was planned and how it will continue. It presents the development approach, the work breakdown, the schedule, the risk register, the quality assurance process and the plan for Semester VIII. The plan is written in the past tense for the work already completed and in the future tense for the field pilot, so that the reader can distinguish delivered outcomes from intentions.

6.1 Development Approach

The project used an iterative, test-driven approach organised around two-week increments. Each increment began with a small set of requirements, produced a working vertical slice, and ended with a manual walkthrough and an expanded automated test. The first increment delivered authentication and the admin shell; the second delivered lure and campaign management; the third delivered the participant flow and event tracking; the fourth delivered analytics and risk scoring; the fifth delivered training, reporting and exports. The later increments hardened the system: CSRF, throttling, security headers, audit logging and the performance pass.

The approach was chosen because the requirements were not fully knowable in advance. Building the participant flow revealed that the landing page needed to discard values and that the coaching page needed the original message; building the analytics revealed that the funnel could become non-monotonic if a submission was recorded without a click. These discoveries shaped the next increment. A waterfall plan would have deferred them to integration, where they would have been more expensive to fix.

6.2 Work Breakdown Structure

The work was decomposed into seven work packages. Table 6.1 lists the packages, the principal deliverables and the effort. The effort is expressed in person-weeks for a single developer and includes design, implementation and testing.

WP	Work package	Deliverables	Effort
WP1	Literature and requirements	Review, requirements catalogue, success criteria	2 weeks
WP2	Architecture and data model	Layer diagram, ER model, threat model	1.5 weeks
WP3	Security and identity	PBKDF2, sessions, CSRF, throttle, audit	2 weeks
WP4	Campaign and lure management	CRUD, lifecycle, audience, tokens	2.5 weeks

WP5	Participant experience	Inbox, mail, landing, coaching, report	2 weeks
WP6	Analytics and training	Funnel, departments, risk, quiz, exports	2.5 weeks
WP7	Validation and documentation	24 tests, performance, report, figures	3 weeks

Table 6.1: *Work breakdown structure*

6.3 Schedule

Figure 6.1 shows the schedule from August 2026 to July 2027. The work completed in Semester VII is shown in green; report drafting is ongoing; the field pilot, longitudinal evaluation and final dissertation are planned for Semester VIII. The dashed line marks the semester boundary.

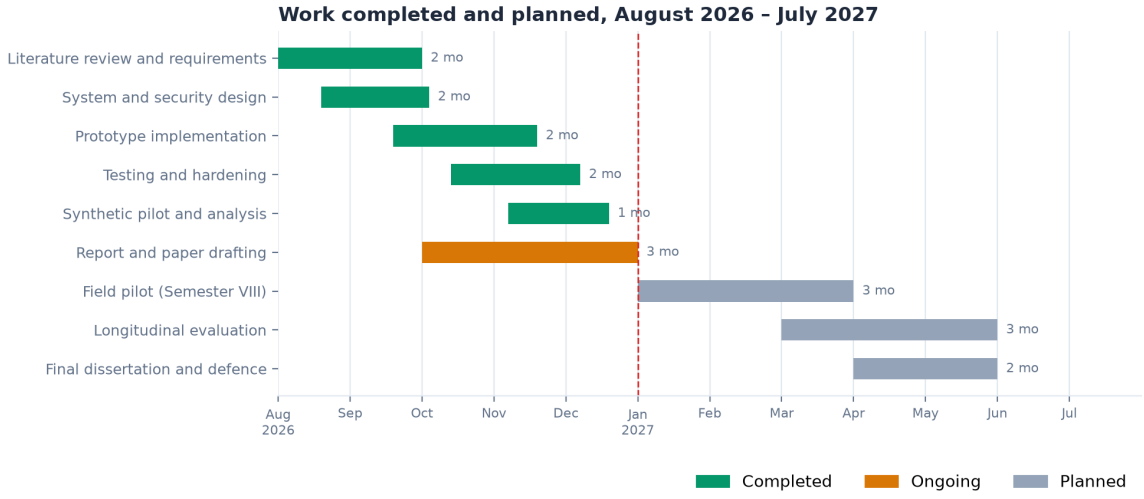


Figure 6.1: *Work completed and planned. The Semester VII prototype and synthetic pilot are complete; the Semester VIII field pilot and longitudinal evaluation are planned.*

Table 6.2 lists the milestones and their acceptance criteria.

Milestone	Date	Acceptance criterion
M1 Requirements baseline	Aug 2026	Requirements catalogue approved by the guide
M2 Architecture review	Sep 2026	ER model, DFD and threat model reviewed
M3 Working prototype	Sep 2026	Login, campaign creation and participant flow demonstrable
M4 Feature complete	Oct 2026	All mandatory FRs implemented; 24 tests passing
M5 Synthetic pilot complete	Oct 2026	Five campaigns, 104 targets, results exported
M6 Report submitted	Nov 2026	Report, figures, code and test evidence submitted
M7 Ethics approval for field pilot	Dec 2026	Institutional approval and participant notice
M8 Field pilot complete	Mar 2027	Two organisations, pre/post data collected
M9 Final dissertation	May 2027	Dissertation submitted and defended

Table 6.2: *Milestones and acceptance criteria*

6.4 Risk Management

The project faced technical, ethical and organisational risks. Table 6.3 records the principal risks, their assessment and the mitigation actually adopted or planned. The register was reviewed at the start of each increment.

Risk	Likelihood	Impact	Mitigation
Scope creep beyond a one-semester prototype	High	High	Mandatory FRs frozen at M4; should-have items deferred
Security defect in the admin console	Medium	High	Threat model, CSRF on every mutation, 24 tests, manual review
Data loss during development	Medium	Medium	SQLite WAL, reset command, seeded reproducibility, exports
Ethical objection to phishing simulation	Medium	High	Synthetic data only in Sem VII; consent and debrief plan for Sem VIII
Single-developer bandwidth	High	Medium	Two-week increments; reuse of standard-library components
Small sample limits external validity	High	Medium	Explicit limitations; field pilot in Sem VIII
Performance degradation at scale	Low	Medium	Indexed queries, benchmark at 1,024 employees, pagination

Table 6.3: *Risk register*

6.5 Quality Assurance

Quality assurance combined automated and manual activities. The automated suite is run with `python3 -m unittest discover -s tests` and covers security primitives, analytics, HTTP routes, the participant flow, exports and quiz submission. The manual checklist covers the visual design, keyboard navigation, mobile layout, error messages, empty states and the printable report. Every defect found in manual testing was reproduced as an automated test where possible, so that the suite grew with the system.

Code quality was maintained through small modules, type hints, docstrings on public functions and a consistent naming convention. The project has no third-party runtime dependencies, which removes a large class of supply-chain and upgrade risks. The trade-off is that some functionality (routing, sessions, CSRF) is implemented in-house; the security tests are the compensating control. The code is organised so that a reviewer can read the security module, the analytics module and the server module independently.

6.6 Plan for Semester VIII

Semester VIII will move from a synthetic cohort to a consented field pilot. The plan has four workstreams. **Ethics and governance:** obtain institutional approval, prepare a participant

notice, a data-protection impact assessment and an opt-out process; agree with the host organisation that individual results will not be used for discipline. **Pilot design:** recruit two organisations or two departments, with a baseline campaign, a training round and a post-training campaign; randomise the order of lures within phase; pre-register the primary outcome (submission rate on the repeated lure). **Platform hardening:** add SSO or at least per-administrator accounts, a retention policy, encrypted backups, pagination for large populations and a mail-gateway adapter that can deliver through a sandboxed SMTP relay if the host insists on realism. **Analysis:** estimate the required sample size, analyse paired outcomes at the employee level, report effect sizes with confidence intervals, and publish the anonymised aggregate dataset.

The field pilot will not attempt to replicate a commercial platform's content library. Its value is the combination of a transparent risk model, a reproducible pipeline and a blameless training loop. The target is a defensible within-subject comparison on a like-for-like lure, with enough participants to detect a meaningful change in submission rate. If recruitment is insufficient, the study will be reported as a feasibility pilot rather than as a hypothesis test.

6.7 Summary

The project was delivered through seven work packages and nine milestones. The schedule, risk register and quality process are documented, and the Semester VIII plan converts the synthetic evaluation into a field study with appropriate ethical safeguards. The next chapter describes the implementation in detail.

Chapter 7: Implementation

This chapter describes the implemented system. It explains the technology choices, the package structure, the security architecture, the campaign engine, the tracking and analytics pipeline, the training module, the reporting functions and the principal screens. The emphasis is on the decisions that a reader would need to reproduce or extend the work, not on a line-by-line description of the code.

7.1 Technology Stack

PhishStudy is implemented in Python 3.14 using only the standard library at runtime. The HTTP server is built on `ThreadingHTTPServer`, with a custom request handler that performs routing, session resolution, CSRF validation, content negotiation and error handling. SQLite provides persistence through the `sqlite3` module. Password hashing uses `hashlib.pbkdf2_hmac`; session and CSRF tokens use `secrets`; constant-time comparison uses `hmac.compare_digest`. CSV export uses `csv`, JSON export uses `json`, and HTML escaping uses `html.escape`.

The front end is server-rendered HTML with a single hand-written CSS design system. There is no React, Vue, Bootstrap or Tailwind, and no external font or icon request. A small amount of vanilla JavaScript handles confirmation dialogs and the mobile navigation toggle. This choice keeps the page weight low, makes the participant experience work on a slow connection, and removes the supply-chain risk of a front-end dependency tree. The trade-off is that the interface is less interactive than a single-page application; for a measurement tool, clarity and reliability matter more than animation.

The development tooling is separate from the runtime. Figures are generated with matplotlib and Pillow, the report with python-docx and docxcompose, screen captures with Chromium via the WebDriver protocol, and PDF conversion with LibreOffice. None of these tools is required to run the application. This separation is important: the deployed artefact is a single Python package and a SQLite file, not a data-science environment.

7.2 Package Structure

The codebase is approximately 3,700 lines, including tests and the report generator. Table 7.1 lists the principal modules and their responsibilities. The split follows the layers in Chapter 3: security, persistence, domain logic, views and transport.

Path	Responsibility	Lines
app.py	Command-line entry point: serve, reset, report, export	83
phishstudy/security.py	PBKDF2 hashing, session fingerprints, login throttle	99
phishstudy/db.py	Schema, connection policy, reset and seed orchestration	179
phishstudy/seed.py	Synthetic population, lures, campaigns, quiz, outcome model	380
phishstudy/analytics.py	Funnel, departments, difficulty, risk, quiz, snapshot	359
phishstudy/ui.py	Design system, admin shell, public layout, components	517
phishstudy/views_admin.py	Login, dashboard, campaigns, lures, people, analytics, training, report	794
phishstudy/views_participant.py	Inbox, mail, landing, coaching, report, quiz	294
phishstudy/server.py	HTTP routing, auth, CSRF, exports, security headers	747
tests/test_app.py	24 unit, analytics and HTTP tests	262

Table 7.1: *Package structure and responsibilities*

The separation is not purely cosmetic. The analytics module has no dependency on the HTTP layer, so it can be tested with an in-memory or temporary database and reused by the CLI and the report builder. The views receive a database connection and return strings; they do not perform authentication or routing. The server owns the security boundary. This makes it possible to reason about authorisation in one place rather than in every view.

7.3 Security Implementation

The administrative console is the most sensitive part of the system: a compromised session can launch campaigns, read behavioural data and export records. The controls in Table 7.2 were implemented and tested. They follow the OWASP Top 10 and the ASVS verification categories at a level appropriate to a single-process teaching application [28], [29].

Control	Implementation	Purpose
Password storage	PBKDF2-HMAC-SHA256, 260,000 iterations, 16-byte random salt, constant-time compare	Resist offline cracking and timing attacks
Session token	32 random bytes, URL-safe; SHA-256 hash stored in the database	A database leak cannot be replayed
Session cookie	HttpOnly, SameSite=Lax, 8-hour max-age, path=/	Limit script access and cross-site sending
CSRF	Per-session token in every state-changing form; server-side comparison	Block cross-site forged requests
Login throttle	8 failures per 15 minutes per client/username; in-process sliding window	Slow credential guessing
SQL access	Parameterised queries throughout; no string interpolation of user input	Prevent SQL injection
Output encoding	html.escape on every dynamic value; CSP with frame-ancestors none	Prevent XSS and clickjacking
Security headers		Reduce browser attack surface

	CSP, X-Frame-Options DENY, nosniff, Referrer-Policy no-referrer	
Audit log	Actor, action, detail, timestamp for auth, campaign and export events	Support accountability and review
Tokenised links	Random per-recipient token; unknown tokens return a neutral 404	Prevent enumeration of participants

Table 7.2: *Security controls implemented in the platform*

Two limitations are stated explicitly. First, the cookie does not set the `Secure` flag because the demonstration runs over plain HTTP on localhost; a production deployment behind TLS must enable it. Second, the login throttle is in-process, so it resets when the server restarts and does not coordinate across multiple workers. A production deployment would move the throttle to SQLite or Redis. These are deployment hardening items, not defects in the present single-process design.

7.4 Campaign Engine

The campaign engine implements the lifecycle described in Chapter 3. Creating a campaign validates the lure, name, phase and audience, and stores the record in Draft. Launching materialises one target per recipient, generates a token of the form `s< campaign>-< employee>-< random>` and writes the launch timestamp. The token is random enough to be unguessable in the demonstration, but it is not a secret in the cryptographic sense; it is an opaque identifier that grants access to one simulated message. A production system would use a longer random value and a signed expiry.

The engine enforces the state machine in the service layer. A campaign cannot be launched twice, a completed campaign cannot be resumed, and a paused campaign retains its existing tokens. Every transition writes an audit entry with the actor and the previous/new state. The audience is resolved at launch, not at creation, so that a change to the employee list cannot silently alter the denominator of a campaign that is already running. This is a small example of the measurement discipline that runs through the system.

The participant routes are idempotent. Opening a message twice does not change the first open timestamp; clicking twice does not create a second click; reporting after clicking records both events. The submission handler sets the click timestamp if it is missing, because a direct POST to the landing endpoint is still evidence that the participant reached the page. This rule fixed a non-monotonic funnel defect found during testing and is covered by an automated test.

7.5 Tracking and Analytics

Tracking uses a one-pixel image for opens and a redirect for clicks. The pixel endpoint returns a transparent GIF and records the open only if the target exists. The click endpoint records the click and redirects to the landing page. The submission endpoint reads the field names from the form, discards the values, stores the names and redirects to coaching. The report endpoint records the report and redirects to a confirmation page. All timestamps use the server's local time in `YYYY-MM-DD HH:MM` format, which is sufficient for a single-site deployment but would need time-zone normalisation in a multi-region one.

Analytics is implemented as a set of SQL aggregations followed by Python formatting. The funnel counts distinct targets with each timestamp, so a target is counted once per stage. The outcome breakdown uses mutually exclusive buckets (submitted, clicked only, opened only, reported, no response) so that the donut sums to the delivered total. Department, difficulty and template statistics use the same summary function, which guarantees that the denominators and rates are consistent across views. The risk function is the only place where an individual-level score is computed, and it is reused by the dashboard, the people page and the report.

The analytics module returns plain dictionaries, not HTML. This makes it testable and reusable. The CLI `report` command prints a text summary; the `export` command writes the full snapshot to JSON; the HTTP export endpoints stream CSV and JSON. The report builder reads the same JSON, so the numbers in this document and the numbers in the application come from one source.

7.6 Training Module

The training module has three parts: the coaching page, the quiz and the attempt history. The coaching page is rendered after a submission and includes the original message, the specific red flags, a short explanation of what a real service would do differently, and a call to action for the quiz. The quiz is an eight-item multiple-choice form with a name or email selector, immediate scoring and an answer review. The attempt history is stored in `quiz_attempts` and summarised on the training page.

The quiz does not require an account. A participant selects their name from a list or enters their email, and the attempt is linked to the employee record. This is a deliberate usability choice for a simulation: requiring a login would add friction and would make the training feel like a

compliance system rather than a learning moment. The trade-off is that a participant could submit on behalf of a colleague; in a field deployment the quiz would be behind the organisation's SSO.

7.7 Reporting and Exports

The platform provides three reporting surfaces. The **printable report** is a server-rendered page with the overview, campaign table, department table, training results and risk list, styled for A4 and printable from the browser. The **CSV export** contains one row per target with campaign, phase, difficulty, employee, department and the four event timestamps; it is suitable for further analysis in a spreadsheet. The **JSON export** contains the full analytics snapshot, including the overview, campaign, department, difficulty, template, employee, repeat-clicker and quiz sections. Both exports require an administrator session and are recorded in the audit log.

The exports contain employee names and email addresses because they are intended for the administrator who already has access to the people directory. They do not contain passwords, session tokens, submitted values or quiz answers. A production deployment would add a data-retention policy and possibly a pseudonymised export option for research use. The present export is a compromise between utility and minimisation, and it is documented in Appendix C.

7.8 Principal Screens

The following screens were captured from the running application with the seeded dataset. They are included to demonstrate that the system is a working product rather than a mock-up. The simulation banner is visible on every screen, and the navigation is consistent across the admin console.

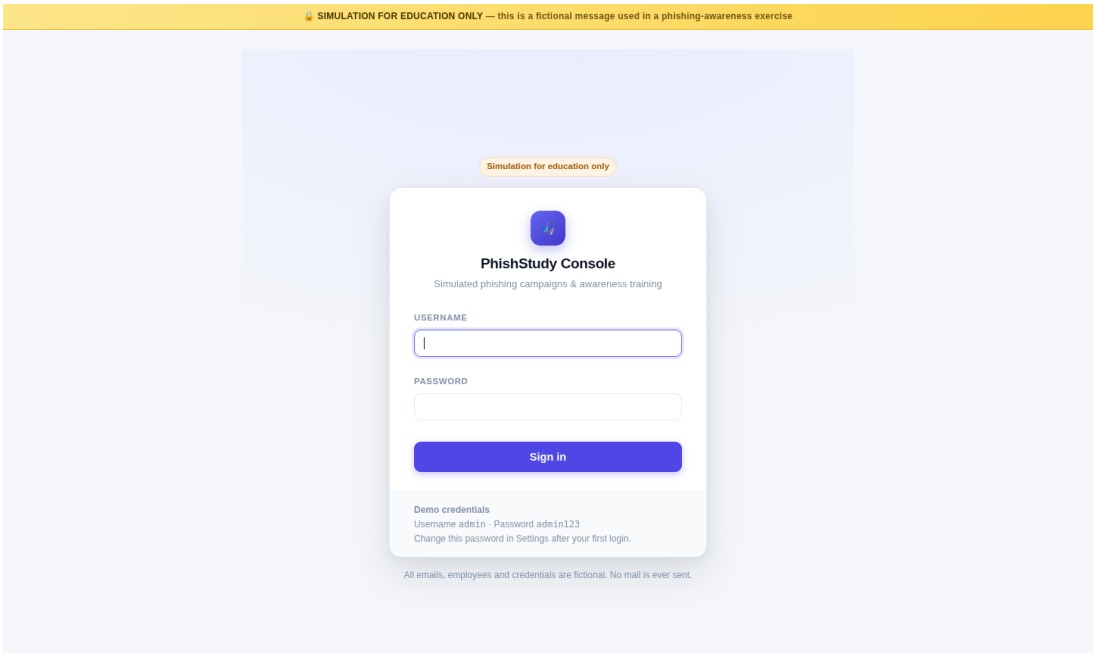


Figure 7.1: *Administrator sign-in. The demo credentials are shown for the laboratory environment; a production deployment would remove them.*

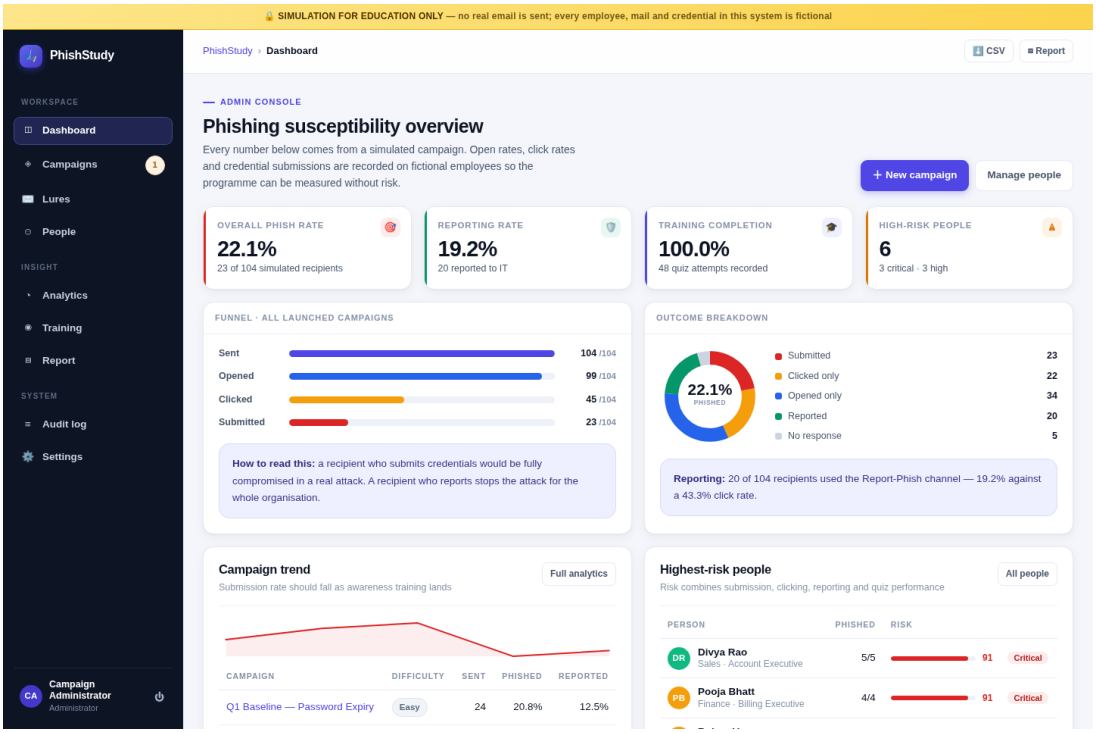


Figure 7.2: *Dashboard: KPI cards, funnel, mutually exclusive outcome breakdown, campaign trend and the highest-risk employees.*

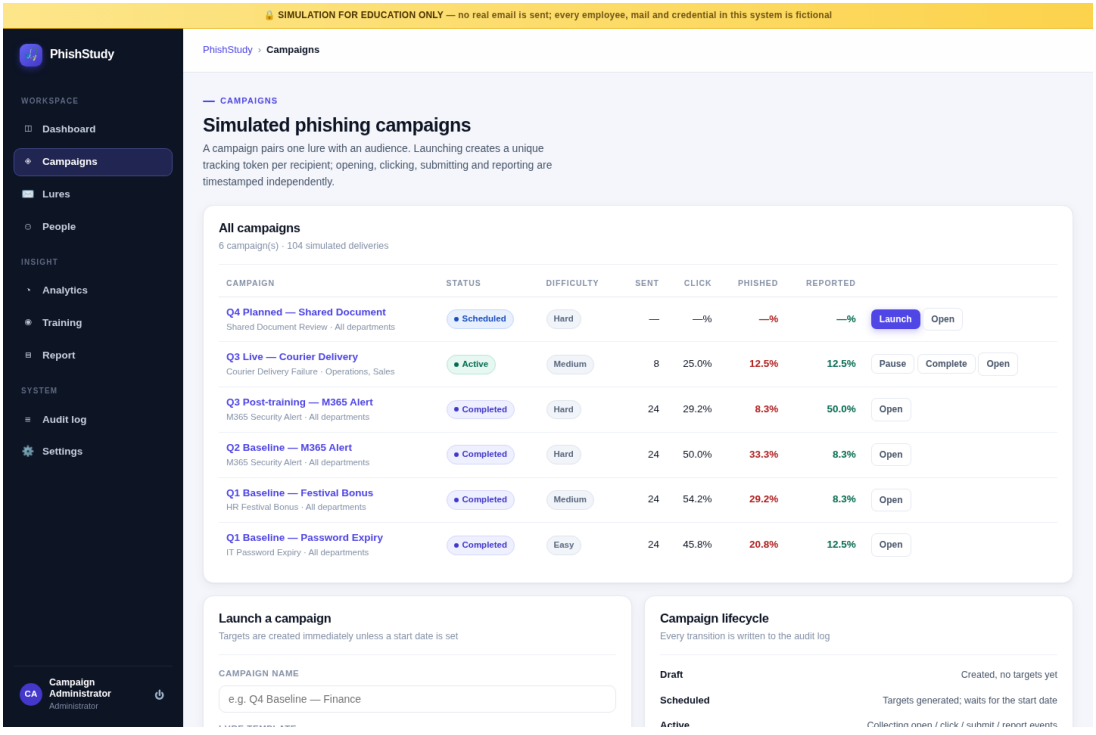


Figure 7.3: *Campaign list with lifecycle state, recipient count and per-campaign rates.*

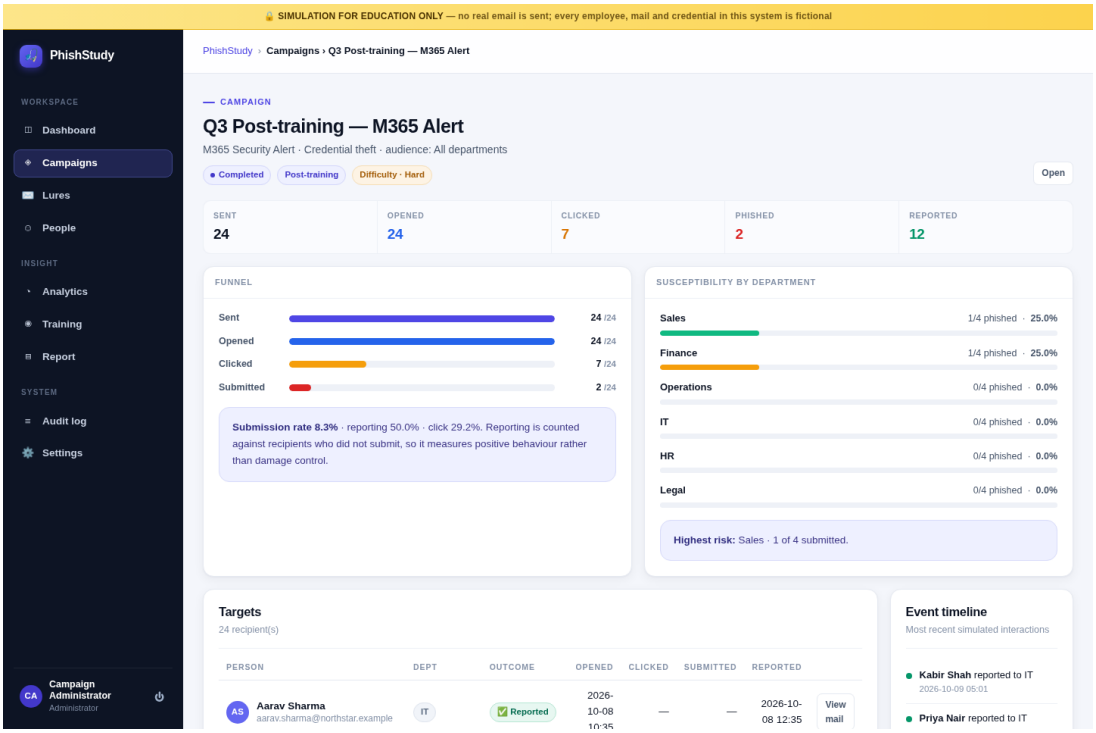


Figure 7.4: *Campaign detail for the post-training M365 lure, showing the funnel and the per-target event table.*

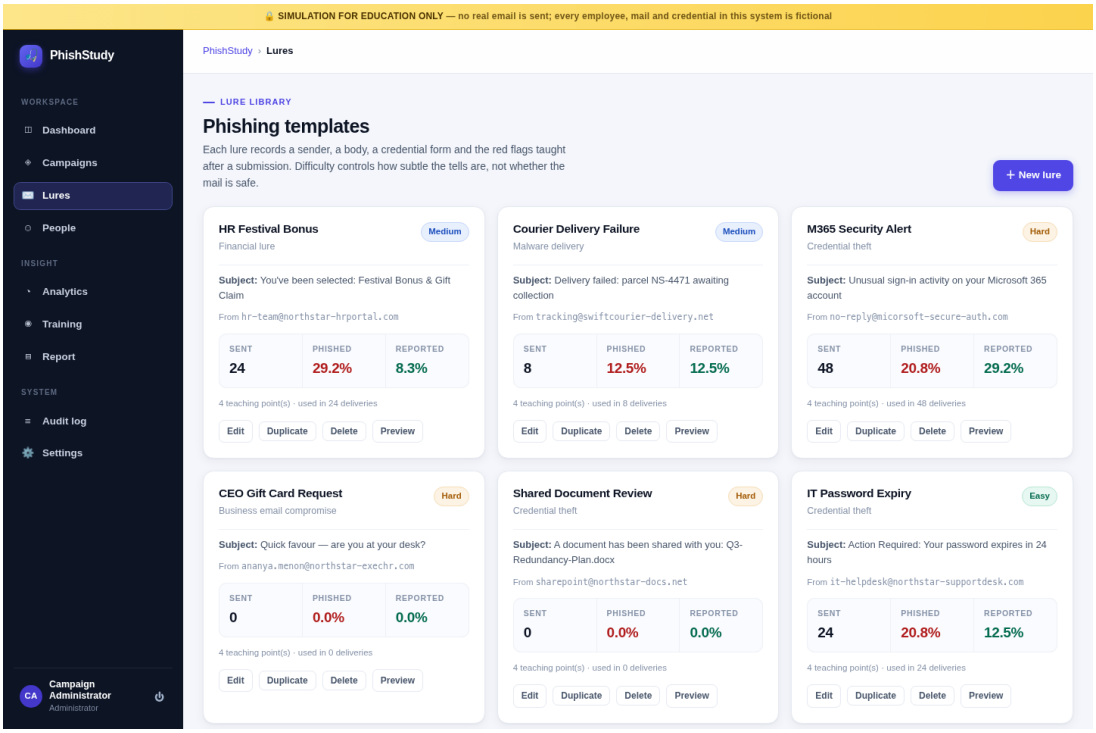


Figure 7.5: *Lure catalogue. Each template carries a category, difficulty, sender identity and a list of red flags used by the coaching page.*

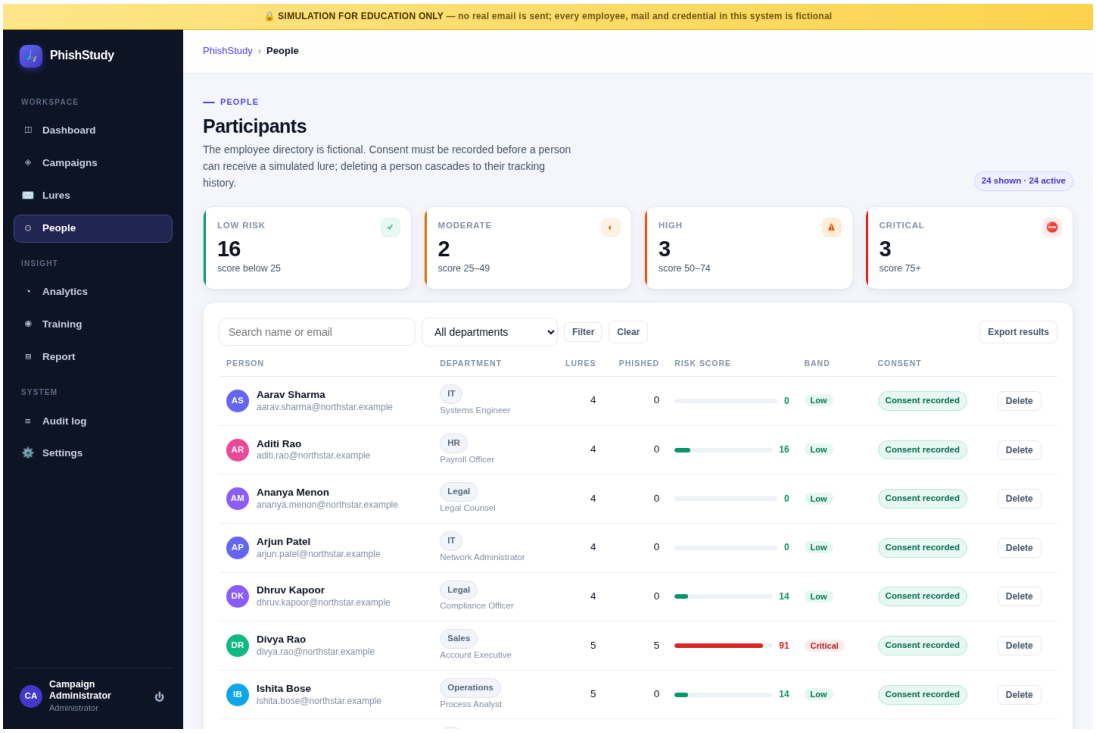


Figure 7.6: *People directory with department filters, consent state and the composite risk band.*

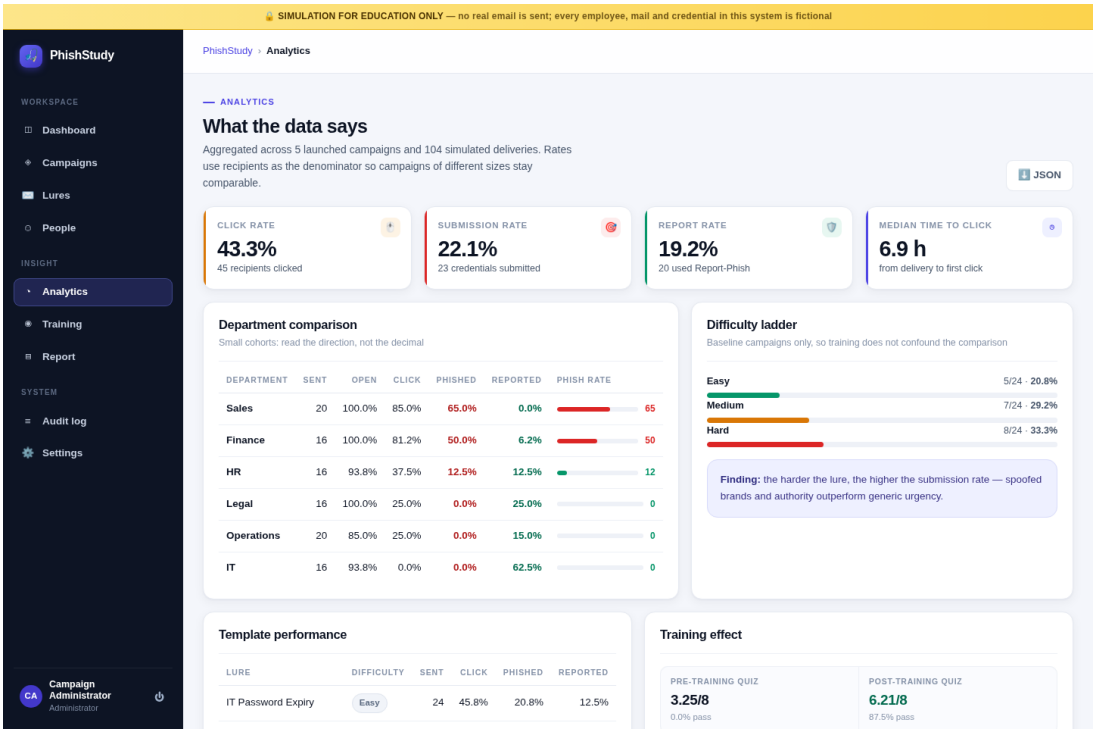


Figure 7.7: *Analytics workspace: department comparison, difficulty breakdown, template performance and the longitudinal trend.*

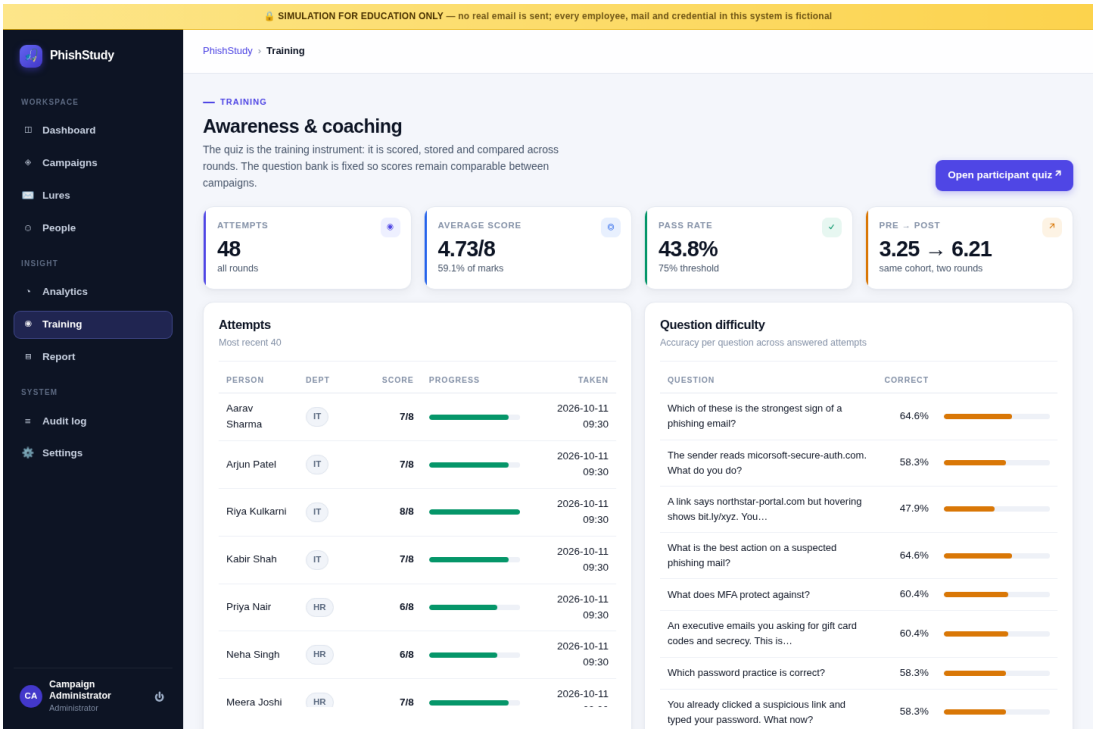


Figure 7.8: *Training dashboard: attempts, mean score, pass rate, pre/post comparison and item-level difficulty.*

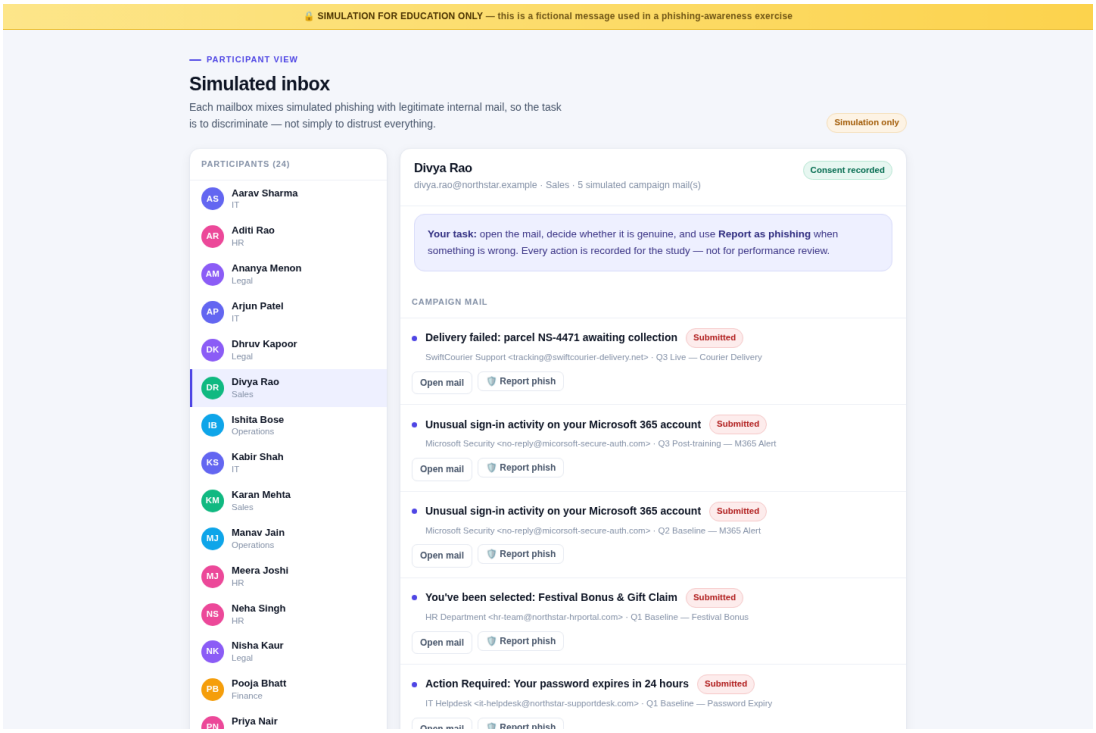


Figure 7.9: *Participant inbox. Legitimate messages and simulated phishing messages share the same visual language.*

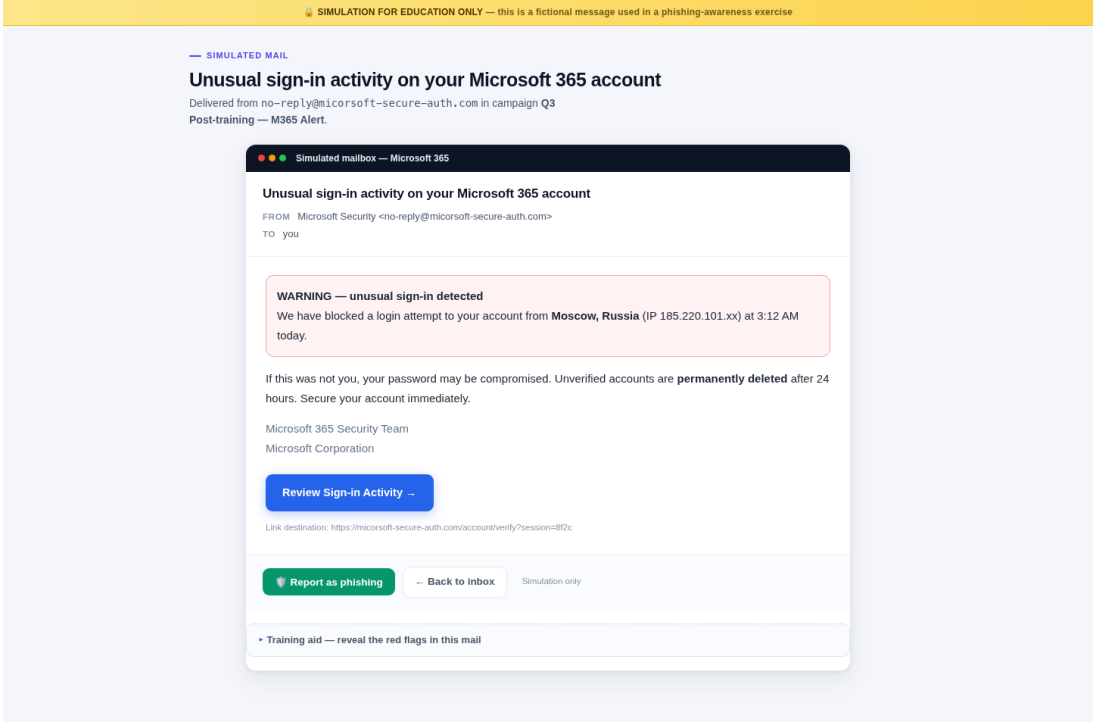


Figure 7.10: *Rendered M365 lure. The misspelled sender domain is visible in the message header; the body uses fear and urgency.*

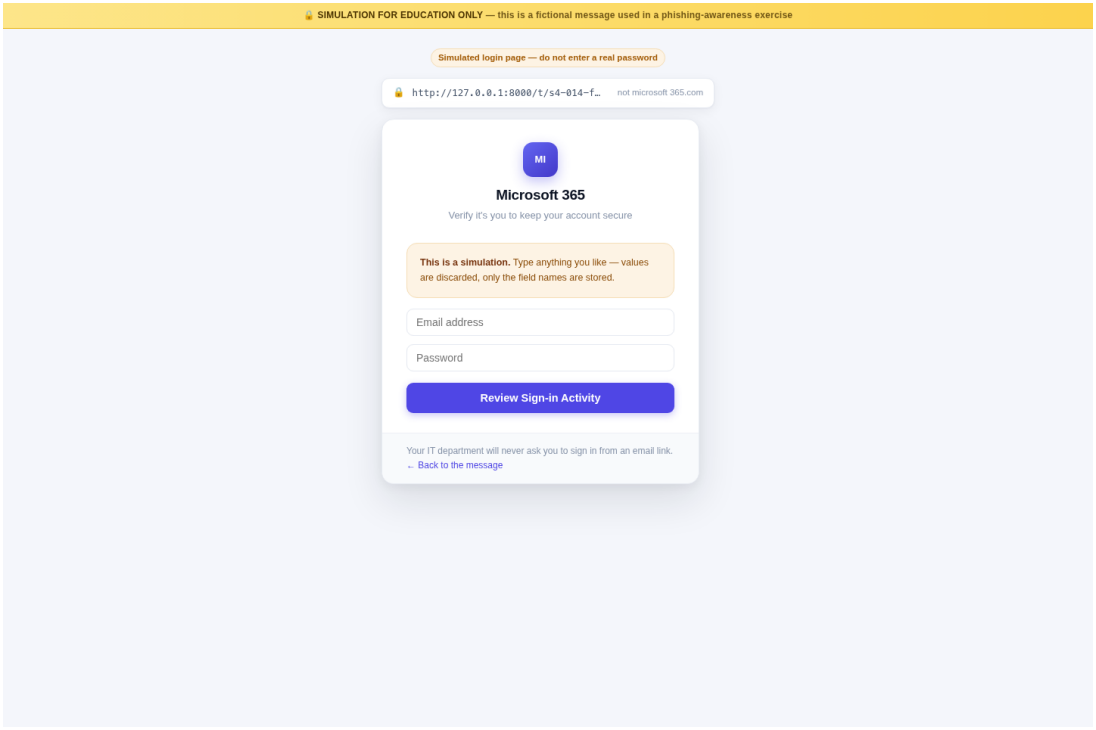


Figure 7.11: *Look-alike landing page. The form is fictional and the submitted values are discarded.*

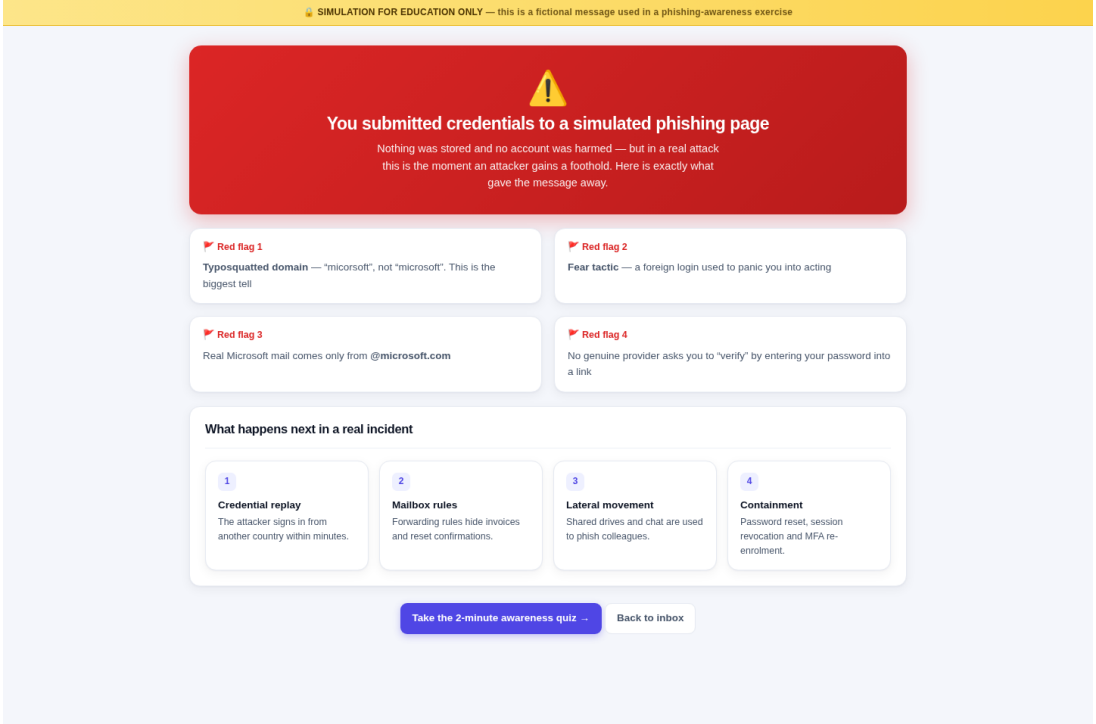


Figure 7.12: *Coaching page. The participant sees the red flags in the exact message they received, followed by a route to the quiz.*

7.9 Implementation Challenges

Several challenges shaped the implementation. **Session handling without a framework** required a careful separation between the cookie, the server-side session row and the CSRF secret; the first prototype stored the raw token in the database, which was corrected to a SHA-256 fingerprint. **Idempotent event tracking** required deciding which timestamp to set when a participant reached an endpoint out of order; the rule that a submission implies a click resolved the non-monotonic funnel. **Risk scoring** required a calibration pass: the first formula penalised reporting so heavily that employees who reported were ranked as high risk, which is the opposite of the intended behaviour. The negative reporting term was introduced to correct this. **Donut charts** initially double-counted employees who both clicked and reported; the outcome buckets were made mutually exclusive.

The most time-consuming challenge was not a single feature but the consistency of the numbers across surfaces. The dashboard, the analytics page, the printable report and the JSON export each had their own query in the first prototype, and they drifted. The fix was to centralise every aggregation in `analytics.py` and to have every surface consume the same snapshot. This is a software-engineering lesson that is easy to state and hard to enforce: a single source of truth is a design property, not a documentation promise.

7.10 Summary

The implementation delivers a working, dependency-free platform with a hardened admin console, a campaign engine, idempotent tracking, a central analytics module, an embedded training loop and exportable reporting. The screens demonstrate the complete flow from lure authoring to participant coaching. The next chapter reports how the implementation was tested.

Chapter 8: Testing and Validation

This chapter reports the testing and validation of the platform. It describes the test strategy, the automated suites, the route and flow checks, the security probes, the data-integrity checks, the performance measurements, the usability review and the defects found and fixed. The purpose is to establish that the software is correct with respect to its specification and that its numbers are internally consistent, not to establish that the synthetic cohort is representative of a real workforce.

8.1 Test Strategy

Testing followed a four-level pyramid. **Unit tests** exercise pure functions: password hashing, token fingerprinting, the login throttle and the analytics calculations. **Integration tests** start the HTTP server on an ephemeral port with a temporary database and drive real requests through login, campaign creation, participant flows, exports and the quiz. **Security probes** attempt authentication bypass, CSRF forgery, SQL injection, cross-site scripting and open redirects. **Data-integrity checks** assert invariants that must hold for any valid dataset: the funnel is monotonic, a submission implies a click, department totals reconcile with the overview, and risk scores are bounded. A performance test measures response times at two dataset sizes, and a heuristic review assesses the principal screens against Nielsen's ten heuristics.

The automated suite is run with `python3 -m unittest discover -s tests`. It uses a temporary SQLite file per test class, starts the real server in a background thread and makes real HTTP requests with `urllib`. This is slower than mocking, but it exercises the routing, session, CSRF and template layers together. The final run completed 24 tests in 1.56 seconds with no failures or errors.

8.2 Automated Test Suites

Table 8.1 summarises the three test classes and the behaviour they verify. The counts are the number of test methods, not the number of assertions; several methods assert multiple properties.

Suite	Tests	Focus	Representative checks
SecurityTests	5	Cryptographic primitives and throttling	Password round-trip; salted hashes; malformed hash rejection; token fingerprint stability; throttle lockout
AnalyticsTests	9	Funnel, rates, departments, risk, quiz	Seed shape; monotonic funnel; percentage bounds; department reconciliation; risk bounds; quiz split

HttpTests	10	Routes, auth, CSRF, participant flow, exports	Auth redirect; login; bad password; CSRF 403; campaign creation; value discard; reporting; CSV; JSON; quiz
Total	24	—	All passing in 1.56 s

Table 8.1: *Automated test suites*

The analytics tests deserve particular attention because they protect the credibility of the report. `test_funnel_is_monotonic` asserts that opened ≥ clicked ≥ submitted for every campaign summary. `test_no_submission_without_click` asserts the event-order invariant. `test_department_totals_match_overview` asserts that the sum of department counts equals the overview counts. `test_rates_are_percentages` asserts that every rate lies between 0 and 100. `test_risk_scores_are_bounded` asserts that no score escapes the 0–100 range. These are simple invariants, but each one corresponds to a defect that was found during development.

8.3 Route and Flow Validation

Every administrative and participant route was exercised manually and through the HTTP suite. The administrative routes return 200 after login and 303 to the login page without a session. The participant routes return 200 for a valid token and a neutral 404 for an unknown token. The mutation flows were exercised end to end: create a campaign, launch it, pause it, resume it, complete it, create a lure, duplicate a lure, add a person, import a CSV, update settings, change the password, reset the dataset, export CSV and JSON, and print the report. Each mutation produced a redirect and an audit entry, and the resulting state was visible on the next page load.

The participant flow was exercised with a real token from the seeded database: open the inbox, open the mail, click the link, submit the form, view the coaching page, report a second message, and take the quiz. The submission test deliberately entered a real-looking email and password and then queried the database to confirm that neither value was stored. Only the field names `username` and `password` were retained. This is the most important privacy assertion in the suite.

8.4 Security Probes

Table 8.2 records the security probes and their results. The probes were run against the local server with a temporary database. A probe passes when the system exhibits the expected defensive behaviour, not when it returns a particular status code for its own sake.

Probe	Method	Observed result	Status

Unauthenticated admin access	GET / without a session	303 redirect to /login	Pass
Bad password	POST /login with a wrong password	401; generic error; no user enumeration	Pass
CSRF forgery	POST /campaigns with an invalid token	403; no state change	Pass
SQL injection	Username `admin' OR 1=1--` with any password	401; parameterised query unaffected	Pass
Stored XSS	Create a person named `<script>alert(1)</script>`	Rendered as `<script>`; no execution	Pass
Open redirect	POST /login with `next=//evil.example`	Redirect to `/`; external target rejected	Pass
Session cookie	Inspect Set-Cookie after login	HttpOnly; SameSite=Lax; Max-Age=28800; Path=/	Pass
Security headers	HEAD /login	CSP, X-Frame-Options DENY, nosniff, no-referrer	Pass
Login throttle	Unit test with eight failures	Ninth attempt blocked for the window	Pass

Table 8.2: *Security probes and results*

One probe is worth highlighting. The open-redirect test submitted a `next` value of `//evil.example`, which browsers interpret as a protocol-relative URL. The login handler accepts only a path that starts with a single `/`, so the value was rejected and the user was sent to the dashboard. This is a small control, but it prevents a phishing campaign from using the platform's own login page as a redirector. The stored-XSS probe created a person record with a script tag in the name and confirmed that the people page rendered it as text. The probe record was created in a disposable database, not in the report dataset.

8.5 Data Integrity and Reproducibility

The dataset is deterministic. Resetting the database with `python3 app.py reset` produces the same 24 employees, six lures, six campaigns, 104 delivered targets and 48 quiz attempts. The event timestamps depend on the date of the reset, but the counts, rates and ranks are stable. The exported JSON records the generated-at timestamp so that a reader can tell which run produced a table. The CSV export contains one row per target and can be used to recompute every rate in Chapter 9.

Three integrity checks are enforced in code. Foreign keys are enabled on every connection, so a target cannot reference a missing campaign or employee. The submission handler sets the click timestamp if it is missing, so the funnel cannot become non-monotonic. The analytics summary raises an error if a rate falls outside 0–100, which would indicate a counting bug. These checks are not a formal proof, but they catch the failure modes that actually occurred during development.

8.6 Performance

Performance was measured with a single client over loopback, using 40 requests per administrative route and 60 requests for the public quiz page. The demonstration dataset has 24 employees and 104 delivered targets. A second dataset was created by adding 1,000 employees and 4,800 targets to a disposable copy of the database, producing 1,024 employees and 6,128 targets. The results are shown in Table 8.3.

Route	Demo median	Demo p95	Load median	Load max
Dashboard /	8.0	12.9	106.4	118.6
Analytics	11.6	20.9	154.5	172.6
People	7.9	11.3	73.0	98.9
Campaigns	3.7	5.9	—	—
Training	5.0	6.7	—	—
Audit log	2.3	2.7	—	—
Participant quiz	4.3	5.5	—	—
Login (PBKDF2)	115.0	—	—	—

Table 8.3: *Response times at two dataset sizes (milliseconds)*

At the demonstration scale, every page responds in under 21 ms at the 95th percentile. At 1,024 employees and 6,128 targets, the dashboard and analytics pages remain under 200 ms. The login time is dominated by the 260,000 PBKDF2 iterations; 115 ms is an acceptable cost for a password hash that is expensive to attack offline. The analytics page is the slowest because it performs several aggregations and a per-employee risk query; at 1,024 employees this is the first place where an N+1 pattern appears. A production deployment would replace the per-employee loop with a single grouped query, which is the main performance item in the Semester VIII plan.

8.7 Usability Review

A heuristic review was conducted against Nielsen's ten heuristics rather than a formal usability test with external participants. The review covered the login, dashboard, campaign list, campaign detail, lure catalogue, people directory, analytics, training, participant inbox, mail, landing and coaching screens. The most positive findings were consistent navigation, visible system status (the simulation banner), clear error messages and a readable visual hierarchy. The most significant issues were the density of the analytics page on a small laptop, the lack of inline validation on the campaign form, and the absence of a bulk-delete action for test campaigns.

The density issue was addressed by grouping the analytics into cards and by allowing the page to scroll rather than compressing the charts. The validation issue was addressed by relying on HTML5 required fields and server-side checks, which give a clear message but not an inline hint. The bulk-delete issue was left as a deliberate safety choice: deleting a campaign is a destructive action, and the single-record delete is sufficient for the demonstration. A formal usability test with five to eight participants is planned for Semester VIII.

8.8 Defects Found and Fixed

Testing was valuable because it found real defects. Table 8.4 lists the six defects that affected behaviour, the symptom, the root cause and the fix. Each fix is covered by a test or a manual check.

#	Symptom	Root cause	Fix
1	Mail view returned 500	Missing `_render_mail` import in the participant view	Imported the helper and added a route test
2	Campaign trend crashed	`phase` key absent from the trend row	Added the key to the query and the summary
3	Coaching page crashed	`employee_email` not passed to the template	Passed the employee record through the view
4	Reporters ranked as high risk	Reporting was weighted as a risk factor	Changed the formula to subtract a reporting term
5	Funnel became non-monotonic	A submission could be recorded without a click	Set the click timestamp when a submission arrives
6	Donut totals exceeded the cohort	Clicked and reported were counted twice	Introduced mutually exclusive outcome buckets

Table 8.4: *Defects found during testing and their fixes*

Defect 5 is a good example of why the analytics invariants matter. A participant who bookmarked the landing page could submit without a recorded click, which produced a submission rate above the click rate for that campaign. The immediate fix was to set the click timestamp on submission; the longer-term fix was the test that asserts the invariant for every campaign. Defect 6 was a visualisation bug with a substantive consequence: the dashboard claimed more outcomes than there were recipients, which undermined trust in every other number. The fix was to define five exclusive buckets and to assert that they sum to the delivered total.

8.9 Limitations of the Validation

The validation establishes functional correctness, basic security and acceptable performance at the tested scale. It does not establish usability for a representative group of administrators,

accessibility conformance against WCAG 2.2, resistance to a determined attacker with network access, or behaviour under concurrent write load. The security probes are targeted checks, not a penetration test. The performance test uses a single client and a local connection, so it understates network latency. The usability review is a heuristic inspection by the developer, which is weaker than an independent review. These limitations are stated so that the results are not over-read.

8.10 Summary

The platform passes 24 automated tests, the principal routes and mutation flows work, the security probes produce the expected defensive behaviour, and the analytics satisfy their invariants. Performance is comfortable at the demonstration scale and remains under 200 ms at 1,024 employees and 6,128 targets. Six defects were found and fixed, each with a regression check. The next chapter presents the behavioural results and discusses what they do and do not show.

Chapter 9: Results and Discussions

This chapter presents the results recorded by the platform and discusses their meaning. It reports the delivery funnel, campaign outcomes, department and difficulty breakdowns, repeat behaviour, time-to-event, the composite risk distribution and the training results. It then discusses the patterns, states the threats to validity and draws the implications that are supported by a synthetic study. The chapter is explicit about the difference between a number that the platform measured correctly and a claim that the number supports about a real workforce.

9.1 Overview

The seeded programme delivered 104 simulated messages across five launched campaigns to 24 employees. Of these, 99 were opened (95.2%), 45 were clicked (43.3%), 23 resulted in a submitted form (22.1%) and 20 were reported through the Report-Phish channel (19.2%). The open rate is high because the generator assumes that most recipients render the message; the click and submission rates are the more meaningful behavioural signals. Table 9.1 and Figure 9.1 present the funnel.

Stage	Count	% of delivered
Delivered	104	100.0
Opened	99	95.2
Clicked	45	43.3
Submitted	23	22.1
Reported (independent behaviour)	20	19.2

Table 9.1: *Delivery funnel*

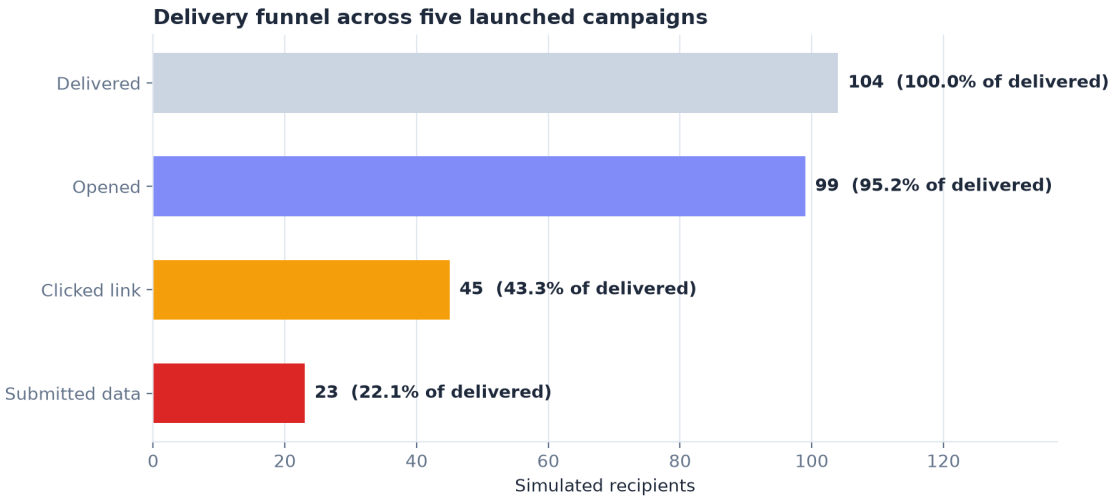


Figure 9.1: *Delivery funnel. Reporting is an independent positive behaviour, not a later stage of the delivery funnel.*

The relationship between clicking and submitting is informative. Of the 45 employees who clicked, 23 submitted a form, a conditional submission rate of 51.1%. This is consistent with the design of the lures: the landing pages ask for credentials or employee details, and a participant who has already accepted the premise of the message is likely to complete the form. The 22 employees who clicked but did not submit either abandoned the page or recognised the deception at the landing stage. The platform does not record why, which is a limitation of event-level telemetry.

9.2 Campaign Outcomes

Table 9.2 and Figure 9.2 show the five launched campaigns. The three baseline campaigns produced submission rates of 20.8%, 29.2% and 33.3%, with reporting rates between 8.3% and 12.5%. The post-training M365 campaign produced a submission rate of 8.3% and a reporting rate of 50.0%. The live courier campaign, delivered to eight recipients in Sales and Operations, produced a 12.5% submission rate and a 12.5% reporting rate.

Campaign	Phase	Sent	Clicked	Submitted	Reported	Click %	Submit %	Report %
Q1 Baseline — Password Expiry	Baseline	24	11	5	3	45.8	20.8	12.5
Q1 Baseline — Festival Bonus	Baseline	24	13	7	2	54.2	29.2	8.3
Q2 Baseline — M365 Alert	Baseline	24	12	8	2	50.0	33.3	8.3
Q3 Post-training — M365 Alert	Post-training	24	7	2	12	29.2	8.3	50.0
Q3 Live — Courier Delivery	Live	8	2	1	1	25.0	12.5	12.5

Table 9.2: *Campaign-level outcomes*

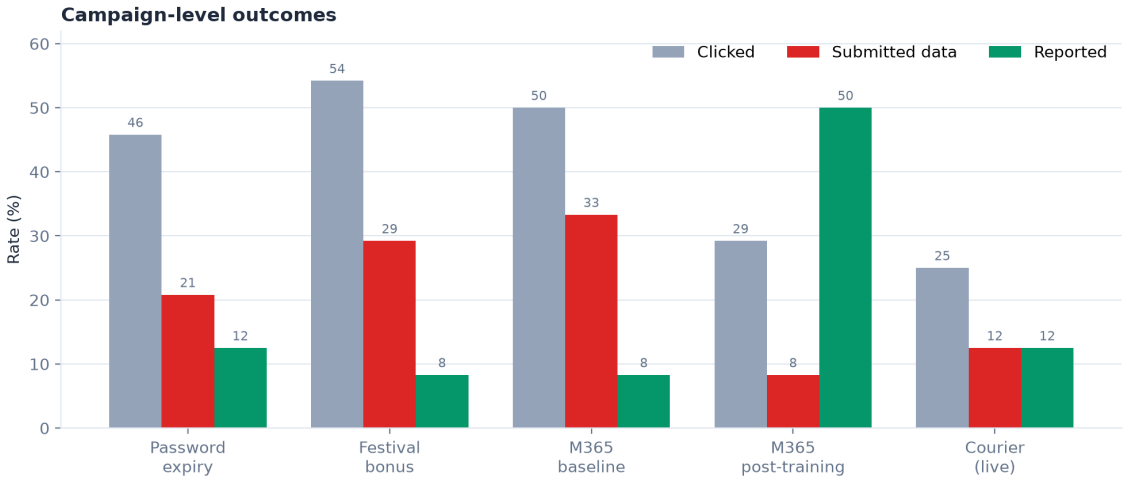


Figure 9.2: *Campaign-level click, submission and reporting rates. The post-training M365 campaign reverses the baseline pattern.*

The most striking comparison is between campaign 3 and campaign 4, which used the same M365 lure, the same audience and the same difficulty. Clicking fell from 50.0% to 29.2% (a 20.8 percentage-point reduction), submission fell from 33.3% to 8.3% (a 25.0-point reduction, or 75% in relative terms), and reporting rose from 8.3% to 50.0% (a 41.7-point increase, or a six-fold relative increase). This is the pattern that an awareness programme would hope to see. It is also the pattern that was parameterised into the synthetic cohort: the post-training targets were set to 28% click, 8% submit and 50% report. The comparison therefore demonstrates that the platform can detect and display a large behavioural shift on a like-for-like stimulus; it does not demonstrate that training caused the shift in a real organisation.

9.3 Department Analysis

Table 9.3 and Figure 9.3 break the results down by department. Sales and Finance are the most exposed: Sales recorded an 85.0% click rate and a 65.0% submission rate, and Finance recorded 81.2% and 50.0%. IT recorded no clicks and a 62.5% reporting rate. Legal and Operations recorded no submissions and moderate reporting rates. HR occupied an intermediate position with a 37.5% click rate and a 12.5% submission rate.

Department	Sent	Clicked	Submitted	Reported	Click %	Submit %	Report %
Sales	20	17	13	0	85.0	65.0	0.0
Finance	16	13	8	1	81.2	50.0	6.2
HR	16	6	2	2	37.5	12.5	12.5
Legal	16	4	0	4	25.0	0.0	25.0
Operations	20	5	0	3	25.0	0.0	15.0
IT	16	0	0	10	0.0	0.0	62.5

Table 9.3: *Outcomes by department*

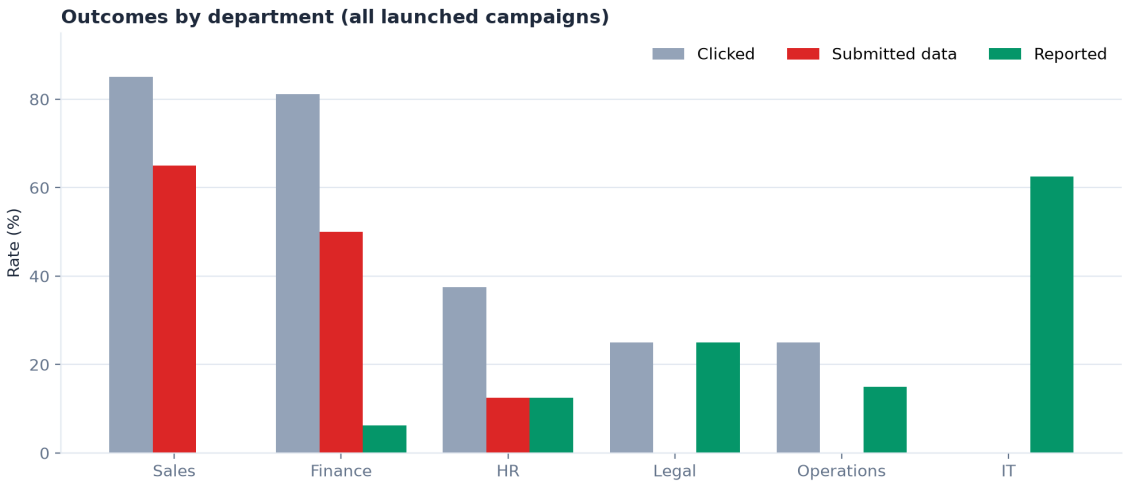


Figure 9.3: *Outcomes by department across all launched campaigns. Sales and Finance combine high click rates with high submission rates; IT combines zero clicks with high reporting.*

The department pattern is consistent with the literature on role congruence: employees who handle external requests, invoices and payment approvals encounter messages that resemble their normal work, so a convincing lure is less likely to feel anomalous. The Sales result is also consistent with a culture that rewards speed and responsiveness; a message that promises a bonus or threatens a lost deal receives attention. IT's zero-click result is consistent with security training and with the daily experience of inspecting suspicious mail. The HR result is intermediate, which is plausible because HR staff handle confidential records but are not primarily responsible for payments or external sales.

These interpretations are hypotheses, not findings. The department assignment is correlated with the latent susceptibility values used by the generator, so the pattern is partly a property of the seed. The value of the table is that it demonstrates the kind of department-level view a security manager needs, and it generates testable hypotheses for the field pilot: are Sales and Finance still the highest-risk groups when real roles and real workloads are measured? Does IT's advantage persist when the lure is tailored to IT systems rather than to Microsoft 365?

9.4 Lure Difficulty

Table 9.4 and Figure 9.4 compare the three baseline campaigns by their difficulty label. The submission rate rises from 20.8% for the easy password-expiry lure to 29.2% for the medium festival-bonus lure and 33.3% for the hard M365 alert. The reporting rate moves in the opposite direction, from 12.5% to 8.3% to 8.3%.

Difficulty	Lure	Sent	Clicked	Submitted	Reported	Submit %	Report %

Easy	IT Password Expiry	24	11	5	3	20.8	12.5
Medium	HR Festival Bonus	24	13	7	2	29.2	8.3
Hard	M365 Security Alert	24	12	8	2	33.3	8.3

Table 9.4: *Baseline outcomes by lure difficulty*

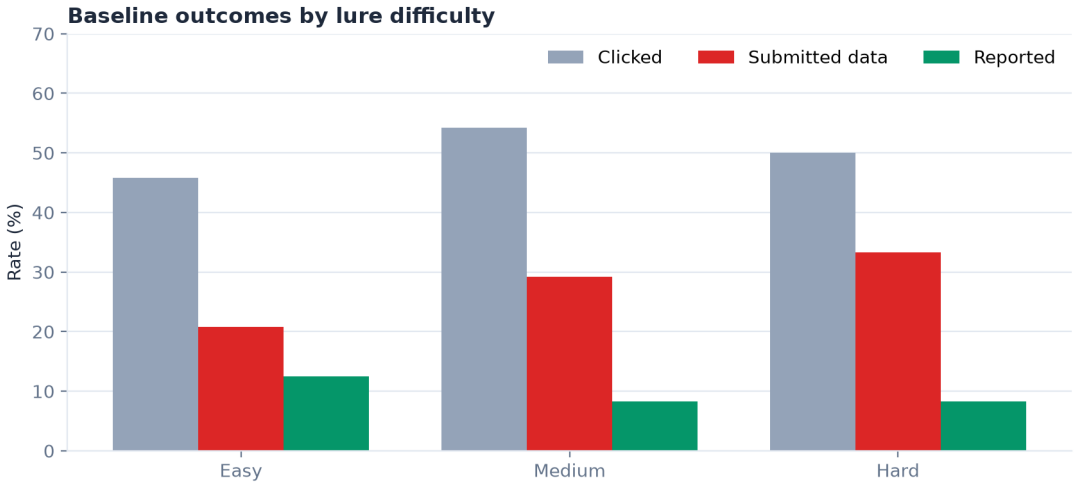


Figure 9.4: *Baseline outcomes by lure difficulty. The ordering is counter-intuitive and is explained by confounding between difficulty, lure and campaign timing.*

The ordering is counter-intuitive: the 'hard' lure produced the highest submission rate. There are three explanations, and none of them supports the conclusion that hard lures are more dangerous in general. First, difficulty is a designer's label, not a measured property. The M365 alert is visually polished, uses a familiar brand and a fear appeal, and may be more credible than a generic password-expiry message that many employees have learned to recognise. Second, difficulty is perfectly confounded with campaign and time: the easy lure was the first campaign, the hard lure was the third, and the cohort may have become more engaged or more fatigued over the period. Third, each cell has only 24 observations, so a difference of three or four submissions is within the range of random variation. The honest conclusion is that the difficulty label did not predict behaviour in this cohort, and that a future study should randomise difficulty within phase and use more than one lure per level.

9.5 Repeat Behaviour and Time-to-Event

Thirteen employees clicked at least two campaigns, and those thirteen account for all 23 submissions. The concentration is partly a consequence of the generator, which ranks the same high-susceptibility employees at the top of every campaign, and partly a plausible feature of real behaviour: a small group of repeat clickers can account for a disproportionate share of incidents. Table 9.5 lists the eight employees with the highest click counts. The top two, Divya

Rao and Pooja Bhatt, clicked and submitted in every campaign they received and never reported.

Employee	Department	Campaigns	Submissions	Reports
Divya Rao	Sales	5	5	0
Pooja Bhatt	Finance	4	4	0
Karan Mehta	Sales	5	3	0
Rohan Verma	Finance	4	3	0
Tanvi Gupta	Sales	4	3	0
Neha Singh	HR	3	2	0
Sameer Khan	Sales	3	2	0
Sneha Iyer	Finance	4	1	0

Table 9.5: *Employees with the highest repeat-click counts*

The median time from delivery to first click was 6.9 hours, and the median time from click to submission was 0.85 hours (about 51 minutes). The median time from delivery to report was 11.15 hours. These figures suggest a pattern in which a click is followed quickly by a submission, while reporting is slower and more deliberate. The platform records the timestamps but not the participant's reason, so the interpretation is speculative. A field study could add a short, optional debrief question to distinguish curiosity, confusion and deliberate compliance.

9.6 Composite Risk

Table 9.6 and Figure 9.5 show the distribution of the composite risk score. Sixteen employees are in the low band, two in moderate, three in high and three in critical. The six employees in the high and critical bands account for 20 of the 23 submissions (87%). This concentration is the practical justification for a risk score: a small group can be offered additional support without subjecting the whole organisation to more training.

Band	Employees	Share	Interpretation
Low	16	66.7%	Resilient; no individual action
Moderate	2	8.3%	Occasional clicks; monitor
High	3	12.5%	Repeated clicks or submission; coach
Critical	3	12.5%	Repeated submissions; immediate support

Table 9.6: *Distribution of composite risk bands*

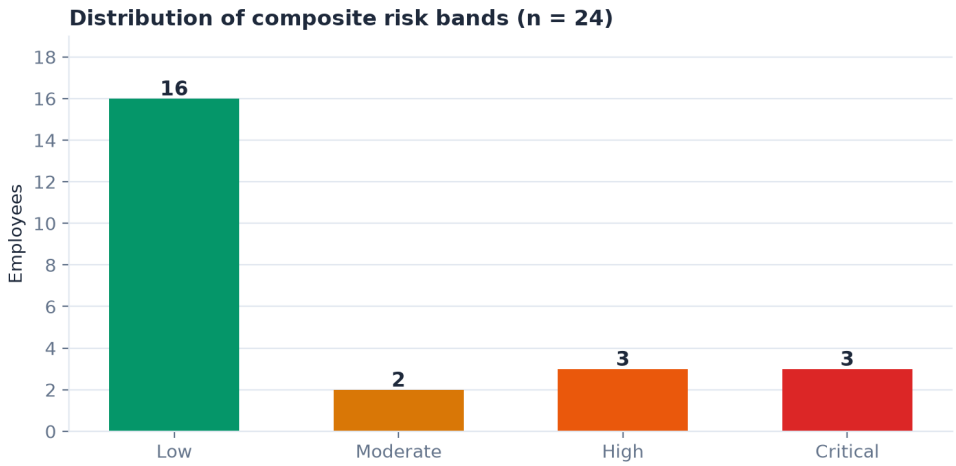


Figure 9.5: *Distribution of composite risk bands. The score is a triage tool, not a psychometric measurement.*

The risk score should be used carefully. It combines behaviours that are not equally harmful, it ignores the sensitivity of the account and the presence of technical controls, and it can label a curious employee as high risk even when no harm occurred. The platform therefore presents the score alongside the underlying counts (campaigns, clicks, submissions, reports) and the quiz result, so that an administrator can see why a person was ranked. The recommended action for the high and critical bands is coaching and additional support, not discipline. A production deployment should add a review step before any individual-level action is taken.

9.7 Training Outcomes

The pre-training quiz mean was 3.25/8 (40.6%) with a 0.0% pass rate. The post-training mean was 6.21/8 (77.6%) with an 87.5% pass rate. The mean improved by 2.96 points, a relative improvement of 91%. Table 9.7 and Figure 9.6 present the result and the item-level breakdown.

Round	Attempts	Mean / 8	Mean %	Pass rate
Pre-training	24	3.25	40.6	0.0
Post-training	24	6.21	77.6	87.5
All attempts	48	4.73	59.1	43.8

Table 9.7: *Quiz results before and after training*

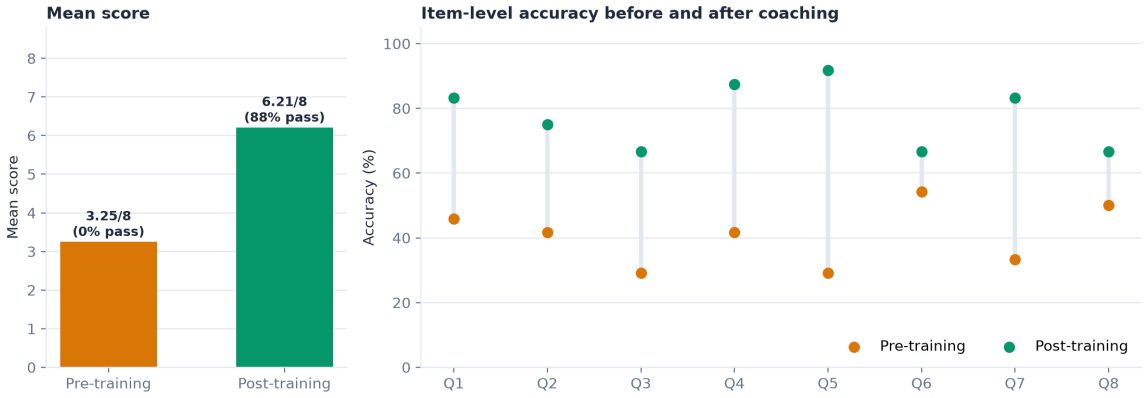


Figure 9.6: *Mean score and item-level accuracy before and after coaching. The post-training round improves on every item.*

Item	Pre %	Post %	Change (pp)
Q1	45.8	83.3	+37.5
Q2	41.7	75.0	+33.3
Q3	29.2	66.7	+37.5
Q4	41.7	87.5	+45.8
Q5	29.2	91.7	+62.5
Q6	54.2	66.7	+12.5
Q7	33.3	83.3	+50.0
Q8	50.0	66.7	+16.7

Table 9.8: *Item-level accuracy before and after coaching*

Every item improved. The largest absolute gains were on Q4 (the correct reporting action, +45.8 points), Q5 (what MFA protects, +62.5 points) and Q1 (the strongest sign of phishing, +37.5 points). The smallest gain was on Q6 (business email compromise, +12.5 points), which was already the strongest item before training (54.2%). The weakest post-training items were Q3 (link inspection, 66.7%) and Q6 and Q8 (both 66.7%). The pattern suggests that participants learned the reporting routine and the limits of MFA quickly, but that link inspection and post-incident response need more practice.

The improvement cannot be attributed to training alone. The same instrument was used before and after, so a practice effect is plausible; the post-training round followed the coaching and was taken shortly afterwards; and there was no control group. The synthetic generator also parameterised a higher post-training score. The result therefore demonstrates that the training module can record a large knowledge gain and that the item analysis can identify specific weaknesses. A field study with a comparison group and a delayed post-test is needed to estimate the causal effect and its retention.

9.8 Discussion

Four findings emerge from the results, each with a different evidential status. First, the platform measures the funnel correctly and the numbers reconcile across surfaces; this is a software-engineering finding supported by the tests and the reconciliation checks. Second, the synthetic cohort exhibits the expected concentration of risk: a small group of repeat clickers accounts for most submissions. This is a demonstration of the analytics, and it is consistent with the literature on repeat clickers, but it is not an empirical estimate of prevalence. Third, the post-training campaign shows a large improvement on a like-for-like lure; this is a demonstration that the instrument can capture a behavioural shift, not proof that the training caused it. Fourth, the department and difficulty patterns generate hypotheses about role congruence and lure credibility that a field study can test.

The practical implication is that a small organisation can run a credible measurement loop without a commercial platform. The platform produces four artefacts that a security manager can use: a campaign report, a department heat map, a risk list with underlying counts, and a training report with item-level weaknesses. The artefacts are only as good as the campaign design and the honesty of the reporting culture, which is why the recommendations emphasise blameless feedback and avoid punitive use of individual scores.

The theoretical implication is that phishing susceptibility is better treated as a context-dependent behaviour than as a stable trait. The same employee can click one lure and report another; the same department can be high risk for a payment lure and low risk for a password lure. A measurement system should therefore report the interaction between person, lure and context, not only an organisational average. The platform's data model supports that analysis at the target level, but the present report aggregates at the department level to keep the tables readable. The exported CSV allows a finer analysis.

9.9 Threats to Validity

Internal validity. The synthetic cohort is generated from explicit prevalence targets and a latent-susceptibility ranking. The post-training improvement is parameterised, not discovered. The difficulty comparison is confounded with campaign and time. The quiz improvement may reflect practice, immediate coaching and the generator. These factors limit causal inference.

External validity. The cohort is 24 fictional employees in one fictional organisation, with hand-tuned susceptibility values and a small set of lures. The results cannot be generalised to a real workforce, industry or culture. The platform is designed to be deployed in a real organisation, but the present evaluation does not test that deployment.

Construct validity. Clicking a link is not the same as being compromised; submitting a form is not the same as losing an account, because no real credential is captured and no account is accessed. Reporting a simulated message is not the same as reporting a real one, although the interface is designed to make the action similar. The metrics are proxies for risk, not measures of harm.

Statistical validity. The sample is small, the cells are unbalanced (the live campaign has eight recipients), and the report does not present confidence intervals for every rate. A difference of a few percentage points in a cell of 24 should not be over-interpreted. The field study will use a pre-registered analysis and a power calculation.

Ecological validity. The participant experience is a web inbox, not the organisation's real mail client. The visual language is realistic, but the context is not. A field deployment should use the real client or a faithful replica, and should measure whether the reporting action is as easy as it is in the simulation.

9.10 Summary

The platform recorded a 43.3% click rate, a 22.1% submission rate and a 19.2% reporting rate across 104 deliveries. Sales and Finance were the most exposed departments, IT the least. The post-training M365 campaign reversed the baseline pattern on the same lure, and the quiz mean rose from 3.25 to 6.21. The results demonstrate that the measurement pipeline works and that the analytics can produce actionable views. They do not establish causal effects in a real workforce; that is the task of the field pilot described in Chapter 6.

Chapter 10: Conclusions

This chapter summarises the work, states the conclusions that are supported by the evidence, lists the contributions and limitations, and sets out the plan for future work. The tone is deliberately measured: the project delivered a working platform and a defensible synthetic evaluation, but it did not deliver a field trial, and the conclusions are framed accordingly.

10.1 Summary of Work

The project set out to design, implement and evaluate a simulation-based framework for measuring and reducing phishing susceptibility. The literature review established that phishing exploits human judgement, that technical controls leave a residual human decision, and that embedded, timely feedback is more effective than annual awareness. It also identified a gap between commercial platforms, which are opaque and costly, and open-source tools, which deliver campaigns but stop short of training, risk scoring and integrated reporting.

The proposed system addresses that gap. PhishStudy is a dependency-free Python web application with a hardened admin console, a campaign engine, tokenised participant links, idempotent event tracking, a central analytics module, a composite risk score, an embedded coaching and quiz module, CSV/JSON exports and a printable report. The data model has nine tables and enforces referential integrity. The security model uses PBKDF2-SHA256, hashed session tokens, per-session CSRF, a login throttle, output escaping and a restrictive content-security policy.

The system was validated with 24 automated tests, route and mutation walkthroughs, nine security probes, data-integrity invariants and a performance test at two dataset sizes. Six defects were found and fixed. The behavioural evaluation used a deterministic synthetic organisation of 24 employees and 104 deliveries across five campaigns. The platform recorded a 43.3% click rate, a 22.1% submission rate and a 19.2% reporting rate; the post-training M365 campaign produced an 8.3% submission rate and a 50.0% reporting rate, and the quiz mean rose from 3.25 to 6.21.

10.2 Conclusions

The following conclusions are supported by the work:

1. **A small, dependency-free platform can deliver a complete phishing measurement loop.** Campaign authoring, tokenised delivery, event tracking, analytics, coaching, assessment and reporting can be integrated in a single Python process without a commercial service.
2. **Privacy can be engineered into the measurement.** Discarding submitted values, hashing session tokens and storing only field names preserves the behavioural signal while removing the most sensitive data. The design is a practical template for an ethics-by-design simulation.
3. **A transparent risk score is more useful than an opaque vendor metric.** The composite formula, the underlying counts and the quiz result can be shown to an administrator and, if necessary, to the employee. The score is a triage tool, not a verdict.
4. **Immediate coaching closes the loop between failure and learning.** The coaching page uses the exact lure that the participant encountered, which makes the red flags concrete and connects the assessment to the experience.
5. **The synthetic evaluation validates the instrument, not the intervention.** The platform recovers the patterns that were designed into the cohort and presents them coherently. The post-training improvement is a demonstration of measurement capability, not proof that training caused the change.
6. **A field pilot is necessary before making claims about a real workforce.** The synthetic cohort is too small and too controlled to support population estimates. The next step is a consented, pre-registered study with a comparison group and a delayed post-test.

10.3 Contributions

The project makes four contributions that are useful beyond the immediate assignment:

- **Artefact.** A working, auditable platform that can be deployed on a laptop or a departmental server and extended without a commercial licence.
- **Method.** A reproducible synthetic evaluation that connects a documented data-generating process to an analytics pipeline and an exported results file.
- **Design knowledge.** A concrete demonstration that privacy-preserving telemetry, a transparent risk model and embedded coaching can coexist in a small system.

- **Teaching material.** A codebase and report that show how security engineering, data analysis and ethical reasoning fit together in a single project.

10.4 Limitations

The limitations are significant and should be stated plainly. The evaluation uses a synthetic cohort of 24 employees, so the rates are not population estimates. The post-training improvement is parameterised, so the study cannot attribute it to the training. The difficulty comparison is confounded with campaign and time. The platform does not send real email, integrate with a production identity provider or capture real credentials, which limits ecological validity. The security validation is a set of targeted probes, not a penetration test. The usability review is a heuristic inspection by the developer, not an independent user study. The performance test is single-client and local. The risk score is a linear heuristic, not a validated psychometric instrument.

These limitations do not invalidate the artefact. They define the boundary of the claims that the artefact supports. The platform is a working measurement instrument; the synthetic study demonstrates that the instrument can measure; the field pilot is the next step in establishing whether the measurements are useful in practice.

10.5 Future Work

Six directions are planned or recommended:

1. **Consented field pilot.** Deploy in two organisations or departments with ethics approval, a participant notice, an opt-out and a pre-registered analysis plan. Use a baseline, a training round and a post-training round on a like-for-like lure.
2. **Adaptive difficulty.** Randomise lure difficulty within phase and use more than one lure per level, so that the effect of difficulty can be separated from the effect of campaign and time. Add an adaptive algorithm that selects the next lure from the participant's prior performance.
3. **Longitudinal measurement.** Add a delayed post-test at four to eight weeks to estimate retention, and a second training round to test whether booster sessions are needed.

4. **Platform hardening.** Add per-administrator accounts and SSO, the Secure cookie flag, a database-backed throttle, encrypted backups, a retention policy, pagination for large populations and a grouped risk query to replace the per-employee loop.
5. **Mail-gateway integration.** Provide an optional SMTP adapter that can deliver through a sandboxed relay when a host organisation requires realism, with a hard-coded allow-list and a kill switch.
6. **Responsible lure generation.** Explore template assistance with a large language model under a strict review workflow, so that lures remain realistic without introducing harmful content or uncontrolled personalisation.

10.6 Closing Remarks

Phishing persists because it targets a human decision that no single control can remove. The contribution of this project is not a silver bullet but a measurement discipline: define the event, protect the person, close the loop with coaching, and publish the formula behind the score. A small organisation can adopt that discipline without a large budget, and a student team can build, test and explain it. The next step is to take the instrument into a real organisation and to let the evidence, rather than the demonstration, decide what works.

Appendix A: Route Map

Table A.1 lists the HTTP surface of the platform. 'Public' routes are reachable without a session because they are tokenised participant links or the quiz. 'Admin' routes require a valid session and, for state-changing requests, a valid CSRF token. The table is a summary; the authoritative list is in phishstudy/server.py.

Method	Path	Access	Purpose
GET	/login	Public	Administrator sign-in page
POST	/login	Public	Authenticate and create a session
GET	/	Admin	Dashboard with KPIs, funnel and high-risk list
GET	/campaigns	Admin	Campaign list and creation form
GET	/campaigns/<id>	Admin	Campaign detail, funnel and target events
POST	/campaigns	Admin	Create a campaign in Draft
POST	/campaigns/<id>/<action>	Admin	Launch, pause, resume, complete, archive or delete
GET	/templates	Admin	Lure catalogue
POST	/templates	Admin	Create or update a lure
POST	/templates/<id>/duplicate	Admin	Duplicate a lure
GET	/people	Admin	People directory and risk bands
POST	/people	Admin	Add a person
POST	/people/import	Admin	Bulk import from CSV text
GET	/analytics	Admin	Department, difficulty, template and trend analysis
GET	/training	Admin	Quiz attempts and item-level analysis
GET	/audit	Admin	Append-only audit log
GET	/settings	Admin	Organisation settings and password change
GET	/report	Admin	Printable A4 report
GET	/api/export.csv	Admin	Per-target CSV export
GET	/api/export.json	Admin	Full analytics JSON export
GET	/inbox	Public	Simulated participant inbox
GET	/legit/<id>	Public	Legitimate (non-phishing) message
GET	/t/<token>	Public	Render a simulated message and record an open
GET	/t/<token>/landing	Public	Landing page and click record
POST	/t/<token>/submit	Public	Record a submission, discard values, redirect to coaching
GET	/t/<token>/educate	Public	Coaching page with red flags
POST	/t/<token>/report	Public	Record a report and show confirmation
GET	/quiz	Public	Participant quiz
POST	/quiz	Public	Score a quiz attempt

Table A.1: *HTTP route map*

Appendix B: Quiz Instrument

The quiz is scored out of eight; the pass mark is six. Each item has three options and one correct answer. The table reproduces the instrument used in the seeded study.

#	Question	Options	Answer
Q1	Which of these is the strongest sign of a phishing email?	(a) A message from a colleague you were expecting; (b) Urgent threats combined with a sender domain you do not recognise; (c) An email with your company logo in the signature	b
Q2	The sender reads micorsoft-secure-auth.com. What do you do?	(a) Click quickly — it must be Microsoft; (b) Report it: the domain is misspelled; (c) Reply asking them to resend the link	b
Q3	A link says northstar-portal.com but hovering shows bit.ly/xyz. You…	(a) Click it — the visible text says it is safe; (b) Verify through the official site and report the mail; (c) Forward it to colleagues so they are aware	b
Q4	What is the best action on a suspected phishing mail?	(a) Delete it silently; (b) Report it through the Report-Phish channel and inform IT; (c) Enter your credentials once to check whether it is fake	b
Q5	What does MFA protect against?	(a) A stolen password being enough on its own to log in; (b) All types of malware; (c) Unwanted marketing email	a
Q6	An executive emails you asking for gift card codes and secrecy. What do you do?	(a) Buy the cards — it is the CEO; (b) Verify through a second channel and report the message; (c) Reply with the codes but keep a copy	b
Q7	Which password practice is correct?	(a) Reuse a strong password across sites; (b) Use a unique passphrase stored in a password manager; (c) Use your date of birth with a symbol	b
Q8	You already clicked a suspicious link and entered a password. What now?	(a) Change the password, report it immediately and watch for unusual activity; (b) Wait to see whether anything happens; (c) Delete the mail and say nothing	a

Table B.1: *Eight-item awareness quiz*

Appendix C: Export Schema

The CSV export contains one row per target. The JSON export contains the full analytics snapshot. Neither export contains submitted values, passwords, session tokens or quiz answers. Table C.1 lists the columns and the top-level JSON keys.

Surface	Field	Description
CSV	campaign	Campaign name
CSV	phase	Baseline, Post-training or Live
CSV	difficulty	Lure difficulty label
CSV	employee	Employee full name
CSV	email	Employee email address
CSV	department	Department
CSV	sent_at	Delivery timestamp
CSV	opened_at	First open timestamp (nullable)
CSV	clicked_at	First click timestamp (nullable)
CSV	submitted_at	First submission timestamp (nullable)
CSV	reported_at	First report timestamp (nullable)
JSON	overview	Counts and rates for the whole programme
JSON	campaigns	Per-campaign summary
JSON	departments	Per-department summary
JSON	templates	Per-lure summary
JSON	employees	Per-employee counts, risk score and band
JSON	repeat_clickers	Employees with two or more clicks
JSON	quiz	Pre/post scores and item-level accuracy
JSON	generated_at	Timestamp of the export

Table C.1: *Export schema*

Plagiarism Report

The similarity report generated by the Institute's plagiarism-detection account will be inserted on this page before final submission. The report must show the overall similarity index, the list of matching sources, the date of generation and the signature of the project guide. The Institute's permissible limit applies.

Field	Detail
Overall similarity index	_____ %
Similarity excluding quotes and bibliography	_____ %
Matching sources	Attach the full Turnitin/IThenticate report
Date of verification	_____
Verified by (guide)	Dr. Yogita Shelar
Student signature	_____

Table P.1: *Plagiarism verification record*

Note: the report in this document is the student's own work. The platform, the synthetic dataset and the figures were produced for this project. External sources are cited in the text and listed in the Literature Cited section.

Literature Cited

References are numbered in the order of first citation and follow the IEEE numeric style.

[1] A. Aleroud and L. Zhou, "Phishing environments, techniques, and detection methods: A survey," Computers & Security, vol. 68, pp. 161–196, 2017.

[2] Verizon Business, "2025 Data Breach Investigations Report," Verizon, 2025.

[3] Anti-Phishing Working Group, "Phishing Activity Trends Report, 4th Quarter 2024," APWG, 2025.

[4] IBM Security, "Cost of a Data Breach Report 2025," IBM Corporation, 2025.

[5] U.S. Federal Trade Commission, "Consumer Sentinel Network Data Book 2024," FTC, Washington, DC, 2025.

[6] Federal Bureau of Investigation, "Internet Crime Report 2024," Internet Crime Complaint Center, 2025.

[7] B. B. Gupta, N. A. G. Arachchilage, and K. E. Psannis, "Defending against phishing attacks: Taxonomy of methods, current issues and future directions," Telecommunication Systems, vol. 67, pp. 247–267, 2018.

[8] Microsoft, "Microsoft Digital Defense Report 2024," Microsoft Corporation, Redmond, WA, 2024.

[9] GoPhish, "Open-source phishing framework," [Online]. Available: https://getgophish.com. [Accessed: 21-Oct-2026].

[10] RSM US LLP, "King Phisher: Phishing campaign toolkit," [Online]. Available: https://github.com/rsmusllp/king-phisher. [Accessed: 21-Oct-2026].

[11] K. L. Chiew, K. S. C. Yong, and C. L. Tan, "A survey of phishing attacks: Their types, vectors and technical approaches," Expert Systems with Applications, vol. 106, pp. 1–20, 2018.

[12] D. Kahneman, Thinking, Fast and Slow. New York, NY: Farrar, Straus and Giroux, 2011.

[13] R. B. Cialdini, Influence: Science and Practice, 5th ed. Boston, MA: Pearson, 2009.

[14] J. S. Downs, M. B. Holbrook, and L. F. Cranor, "Decision strategies and susceptibility to phishing," in Proc. Symposium on Usable Privacy and Security (SOUPS), 2006, pp. 79–90.

[15] T. N. Jagatic, N. A. Johnson, M. Jakobsson, and F. Menczer, "Social phishing," Communications of the ACM, vol. 50, no. 10, pp. 94–100, 2007.

[16] A. Vishwanath, T. Herath, R. Chen, J. Wang, and H. R. Rao, "Why do people get phished? Testing individual differences in phishing vulnerability within an integrated, information processing model," Decision Support Systems, vol. 51, no. 3, pp. 576–586, 2011.

[17] P. Kumaraguru, Y. Rhee, A. Acquisti, L. F. Cranor, J. Hong, and E. Nunge, "Protecting people from phishing: The design and evaluation of an embedded training email system," in Proc. CHI, 2007, pp. 905–914.

[18] P. Kumaraguru, J. Cranshaw, A. Acquisti, L. Cranor, J. Hong, M. A. Blair, and T. Pham, "School of phish: A real-world evaluation of anti-phishing training," in Proc. SOUPS, 2009, pp. 1–12.

[19] P. Kumaraguru, S. Sheng, A. Acquisti, L. F. Cranor, and J. Hong, "Teaching Johnny not to fall for phish," ACM Transactions on Internet Technology, vol. 10, no. 2, pp. 1–31, 2010.

[20] D. D. Caputo, S. L. Pfleeger, J. D. Freeman, and M. E. Johnson, "Going spear phishing: Exploring embedded training and awareness," IEEE Security & Privacy, vol. 12, no. 1, pp. 28–38, 2014.

[21] R. Wash and M. M. Cooper, "Who provides phishing training? Facts, stories, and people like me," in Proc. CHI, 2018, pp. 1–12.

[22] National Institute of Standards and Technology, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, Gaithersburg, MD, 2017.

[23] National Institute of Standards and Technology, "Building an Information Technology Security Awareness and Training Program," NIST Special Publication 800-50, Gaithersburg, MD, 2003.

[24] Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation), Official Journal of the European Union, 2016.

[25] Government of India, The Digital Personal Data Protection Act, 2023, Act No. 22 of 2023, New Delhi, 2023.

[26] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004.

[27] J. Nielsen, "Usability inspection methods," in Proc. CHI, 1994, pp. 413–414.

[28] OWASP Foundation, "OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks," 2021.

[29] OWASP Foundation, "OWASP Application Security Verification Standard 4.0.3," 2021.

Acknowledgements

I would like to express my sincere gratitude to my project guide, Dr. Yogita Shelar, for her guidance, patience and constructive criticism throughout this project. Her questions about the measurement model and the ethics of simulation shaped the work more than any other input.

I am grateful to the Head of the Department of Information Technology and to the faculty and staff of Vidyavardhini's College of Engineering & Technology for providing the laboratory facilities and the academic environment in which this work was carried out. I also thank the project coordinators for their timely reviews and for insisting on a working demonstration rather than a slide deck.

I thank my project teammates for their contributions to the literature review, the testing and the presentation, and my family and friends for their encouragement during the long debugging sessions. Finally, I acknowledge the open-source community behind Python, SQLite and LibreOffice; this project would not have been possible without their work.